

Confidential Transfers on Sui

Draft Version 0.1

Mysten Labs

July 20, 2026

Abstract

We formalize the confidential-transaction mechanism implemented in the accompanying repository as a cryptographic protocol for a public ledger. The construction allows users to convert public coins into encrypted balances, transfer value privately between registered parties, and recover public coins from private balances, while preserving ledger-level verifiability. An optional auditing layer equips a designated set of auditors with decryption capability over every confidential value. The protocol combines Twisted ElGamal encryption [CMTA20] over Ristretto255 [dVGH⁺23] with non-interactive zero-knowledge proofs for balance conservation, ciphertext consistency, range validity, key consistency, and balance re-keying under rotated keys. We describe the participating roles, the on-chain state, and the core transition protocols, and we model security in the universal-composability framework [Can01], proving that the protocol realizes an ideal functionality that captures its integrity and privacy guarantees.

1 Introduction

Confidential transactions aim to reconcile two requirements that are naturally in tension on a public blockchain. On the one hand, the ledger must enforce correctness: no user should be able to create value, overspend a balance, or violate the supply relationship between public and private representations of an asset. On the other hand, transfer amounts and confidential balances should remain hidden from observers. The system implemented in this repository realizes this objective for Sui-based assets by maintaining encrypted balances on-chain and requiring every state transition that posts fresh ciphertexts to be accompanied by efficiently verifiable zero-knowledge proofs; the remaining transitions (public wraps and homomorphic merges of pending deposits) are publicly checkable and carry none.

At a high level, a public coin of type T may be deposited into a backing pool and represented as confidential balance under a user’s public key. Once in private form, value can be transferred between registered accounts without revealing the amount to the ledger. A holder may later withdraw part of a confidential balance back into a public coin. The central invariant is that the total encrypted value represented on-chain remains synchronized with the public assets held in the pool; the corresponding privacy goal is that observers learn no more about confidential balances and private transfers than what is already implied by public wrap and unwrap events together with the visible transaction graph. The protocol also supports an optional auditing layer in which a small, issuer-maintained list of auditors holds a persistent decryption capability over every registered account.

2 Model and Cryptographic Tools

Let $\mathbb{G}$ denote the Ristretto255 prime-order group of order q [dVGH⁺23], and let $G, H \in \mathbb{G}$ be fixed generators with unknown discrete-log relation, where G is the standard base point and H is obtained by hashing a domain-separated label to the curve. Each confidential account is associated with a key pair (sk, pk) , where $\text{sk} \xleftarrow{\$} \mathbb{Z}_q$ and $\text{pk} = \text{sk} \cdot G$.

The protocol uses Twisted ElGamal encryption [CMTA20]. For a message $m \in \mathbb{Z}_q$ and randomness $r \xleftarrow{\$} \mathbb{Z}_q$, encryption under public key pk is defined by

$$\text{Enc}(\text{pk}, m; r) = (C, D) = (rG + mH, r \cdot \text{pk}).$$

The first component is a Pedersen commitment [Ped92] to m with blinding r , and the second component is a *decryption handle* tied to the recipient's public key. The scheme is additively homomorphic: component-wise addition of ciphertexts corresponds to addition of plaintexts and blindings. In addition, the implementation uses the trivial encryption $\text{TrivEnc}(m) = (mH, \mathcal{O})$, where $\mathcal{O}$ is the identity of $\mathbb{G}$, to embed public values into the encrypted domain. Decryption proceeds by observing that

$$C - \text{sk}^{-1} \cdot D = mH.$$

The implementation restricts balances so that the resulting discrete logarithm remains computationally feasible to recover via a baby-step/giant-step procedure over a bounded domain. To represent a 64-bit balance while preserving efficient range checks, the protocol decomposes a value v into four 16-bit limbs,

$$v = v_0 + 2^{16}v_1 + 2^{32}v_2 + 2^{48}v_3, \quad v_i \in [0, 2^{16}),$$

and encrypts each limb independently. We write

$$\text{EA}(\text{pk}, v) = (E_0, E_1, E_2, E_3), \quad E_i = \text{Enc}(\text{pk}, v_i; r_i).$$

We write $\text{EA}(\text{pk}, v; \rho)$ for the same construction with explicit per-limb blindings $\rho = (r_0, \dots, r_3)$. Whenever the system needs a single encryption of the represented value, it homomorphically collapses the four limbs according to

$$\text{Collapse}(\text{EA}) = E_0 + 2^{16}E_1 + 2^{32}E_2 + 2^{48}E_3.$$

The protocol uses a *multi-recipient* variant of Twisted ElGamal solely for the auditor key encapsulation, whose well-formedness is proved by Π_{KC} (below); encrypted amounts use only the single-recipient form of the previous paragraph. For a message $m \in \mathbb{Z}_q$, a blinding $r \xleftarrow{\$} \mathbb{Z}_q$, and a tuple of recipient public keys $(\text{pk}_1, \dots, \text{pk}_n)$,

$$\text{MREnc}((\text{pk}_1, \dots, \text{pk}_n), m; r) = (C, (D_j)_{j=1}^n), \quad C = rG + mH, \quad D_j = r \cdot \text{pk}_j.$$

The Pedersen commitment C is shared across all recipients; only the decryption handles differ. Any recipient j recovers mH by computing $C - \text{sk}_j^{-1}D_j$, and the marginal cost of adding a recipient is a single group element. $\text{MREnc}_j(\cdot)$ denotes the single-recipient ciphertext (C, D_j) induced by projecting $\text{MREnc}(\cdot)$ onto recipient j .

The protocol relies on five Fiat–Shamir [FS87] NIZKs over $\mathbb{G}$, modeled in the security analysis (Section 6.1) as $\mathcal{F}_{\text{NIZK}}$ instances for the relations below; full constructions and proofs are in Appendix A. We write $\Pi(\cdot; s)$ for a proof on statement s whose witness precedes the semicolon.

DDH Π_{DDH} (Appendix A.2, Figure 3) is the Chaum–Pedersen [CP93] proof of knowledge of a shared discrete logarithm,

$$\mathcal{R}_{\text{DDH}} = \{ ((E, F, X, Y); x) : X = xE, Y = xF \}.$$

It enforces every balance-preserving update: a difference ciphertext Δ encrypts zero under $\mathbf{pk} = \text{sk}G$ iff $\Delta.D = \text{sk} \cdot \Delta.C$, certified by $\Pi_{\text{DDH}}(\text{sk}; G, \Delta.C, \mathbf{pk}, \Delta.D)$. An accepted proof on such a Δ , combined with the on-chain check $\mathbf{pk} \neq \mathcal{O}$ enforced at registration and key rotation, forces the encrypted difference to be zero; the full no-inflation argument, including why the nonzero check is essential, is given in the UC realization (Section 6.3). Key rotation uses a batched generalization of this proof, Π_{rekey} (below), to re-key every balance handle at once under the shared witness $w = \text{sk}'\text{sk}^{-1}$.

ElGamal PoK Π_{EG} (Appendix A.3, Figure 4) is a proof of knowledge of an opening of one Twisted ElGamal ciphertext,

$$\mathcal{R}_{\text{EG}} = \{ ((\mathbf{pk}, C, D); (m, r)) : C = rG + mH, D = r\mathbf{pk} \}.$$

Unlike Π_{DDH} in the balance relation, this proof is used purely for extraction by the simulator and requires no nonzero condition: C is a Pedersen commitment, so its opening is unique under the discrete-log assumption, independently of the public key $\mathbf{pk}$.

Ciphertexts sharing one public key admit *witness-folding* (Appendix B): t same-key instances collapse to a single folded proof $\Pi_{\text{EG}}^{\otimes}$ for the conjunction relation

$$\mathcal{R}_{\text{EG}}^{\otimes} = \{ ((\mathbf{pk}, (C_j, D_j)_{j=1}^t); (m_j, r_j)_{j=1}^t) : C_j = r_jG + m_jH, D_j = r_j\mathbf{pk} \text{ for all } j \},$$

whose size $(2\mathbb{G} + 2\mathbb{F})$ is independent of t ; Π_{EG} is the case $t = 1$. Lemma 4 shows the folded proof is an unconditional NIZK proof of knowledge for $\mathcal{R}_{\text{EG}}^{\otimes}$, so it instantiates the corresponding $\mathcal{F}_{\text{NIZK}}$ relation wherever the analysis expects the per-ciphertext proofs. The algorithms in Section 5 use one $\Pi_{\text{EG}}^{\otimes}$ instance per key in every operation carrying multiple same-key ciphertexts.

Range Π_{range} is a proof of knowledge of bounded openings of a list of Pedersen commitments (the limb components $E.C$),

$$\mathcal{R}_{\text{range}}^w = \{ ((C_\ell)_\ell; (m_\ell, r_\ell)_\ell) : C_\ell = r_\ell G + m_\ell H, m_\ell \in [0, 2^w) \}.$$

Knowledge soundness, not plain soundness, is required: Pedersen commitments are perfectly hiding, so $\{C : \exists(m, r), C = rG + mH \wedge m \in [0, 2^w)\}$ is the trivial language $\mathbb{G}$, and only an extractor of the prover’s opening pins a value. We instantiate it with Bulletproofs [BBB⁺18], at $w = 16$ over balance limbs and $w = 32$ over the key-encapsulation limbs below.

Key consistency Π_{KC} (Appendix A.4, Figure 5) is a batched Σ -protocol proving that eight multi-recipient ciphertexts encapsulate the 32-bit limbs of a secret key,

$$\mathcal{R}_{\text{KC}} = \left\{ ((\mathbf{pk}_U, (\mathbf{pk}_{A_j})_j, (C_i, (D_{ij})_j)_i); (u_i, r_i)_i) : \begin{array}{l} C_i = r_i G + u_i H, \\ D_{ij} = r_i \mathbf{pk}_{A_j}, \\ \mathbf{pk}_U = (\sum_i 2^{32i} u_i) G \end{array} \right\}.$$

It simultaneously ties each handle D_{ij} to the blinding of its commitment C_i and certifies that the committed limbs reconstruct sk_U with $\mathbf{pk}_U = \text{sk}_U G$, so an auditor in slot j can later recover sk_U from the stored encapsulation. It is produced once per account, at registration and key rotation; a companion Π_{range}^{32} pins each $u_i \in [0, 2^{32})$ so the reconstructed scalar is unique. Since the eight 32-bit limbs span $256 > \log_2 q$ bits, the relation $\mathbf{pk}_U = \text{sk}_U G$ fixes their reconstruction only modulo q ; the auditor takes the canonical $\text{sk}_U \in [0, q)$ by reducing mod q .

Re-keying Π_{rekey} (Appendix A.5, Figure 6) is a batched Chaum–Pedersen proof of knowledge of a single scalar w that relates the old and new keys and re-keys every balance handle,

$$\mathcal{R}_{\text{rekey}} = \{ ((\text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3); w) : \text{pk}'_U = w \text{pk}_U, D'_i = w D_i \text{ for } i = 0, \dots, 3 \}.$$

Key rotation (Algorithm 8) keeps the four limb commitments $E_i.C$ and replaces only the handles, so the witness $w = \text{sk}'_U \text{sk}_U^{-1}$ carries each old handle D_i to its counterpart D'_i under the new key pk'_U , which the ledger additionally checks is nonzero. As with Π_{DDH} , the check $\text{pk}'_U \neq \mathcal{O}$ ensures that the re-keyed limbs still encrypt the original values.

Definition 1. *An encrypted amount $\text{EA} = (E_0, E_1, E_2, E_3)$ under public key pk is well-formed if each limb plaintext lies in $[0, 2^{16})$ and each limb is a valid Twisted ElGamal encryption under pk . On-chain, the ledger checks this from submitted proofs: a range proof π_{range} for $(E_i.C)_{i=0}^3$ and a single witness-folded consistency proof γ (one $\Pi_{\text{EG}}^{\otimes}$ instance covering the four limbs $(E_i)_{i=0}^3$ under pk), which the holder generates off-chain and includes in the transaction.*

3 Protocol Description

We first describe the parties and state maintained by the protocol, then the transfer and auditing flows. The state transitions of this section appear as pseudocode in Section 5, with variables and primitives explained inline as comments.

3.1 Parties and State

Parties. The protocol involves four kinds of parties. The *ledger* stores all confidential state and verifies the accompanying proofs. The *coin issuer*, the party controlling the public token type T , creates the confidential token object associated with T , maintains the on-chain list of auditor public keys described below, and may impose policy restrictions or emergency controls such as freezing. *Account holders* each register a public key for a given confidential asset and may thereafter hold encrypted balances, receive confidential transfers, and convert between the public and private representations of the asset. *Auditors*, each associated with an encryption key pair $(\text{sk}_{A_j}, \text{pk}_{A_j})$, hold no confidential balance; their sole role is to obtain a decryption capability over the confidential state of every registered account through the mechanism of Section 3.3. A configuration with no auditors is a valid instantiation and reduces to the baseline protocol.

State. For each token type T , the confidential system maintains a shared pool containing the public coins that back all confidential balances. The issuer also maintains an on-chain auditor list $\mathcal{J} = (\text{pk}_{A_1}, \dots, \text{pk}_{A_n})$ scoped to T . Each registered account holder maintains an active encrypted balance, represented as a four-limb encrypted amount under that holder’s public key pk . Upon registration this active balance is initialized to a well-formed encryption of zero, and therefore carries bound counter $\tau_{\text{act}} = 1$ from the outset. Alongside the active balance, the account stores a *verifiable key encapsulation* of the holder’s secret key sk to the auditor list $\mathcal{J}$, produced by the holder off-chain and verified on-chain when the account is created or its key is rotated. In addition, the account stores pending deposits in two forms: a cumulative encrypted amount representing confidential transfers received since the last merge, and a public accumulator representing coins wrapped directly into the account.

Bound counter. To keep merged balances within the bounded discrete-log domain, each encrypted aggregate carries an integer *bound counter* τ that counts the number of **u16**-bounded values folded into it. Each limb plaintext is therefore bounded by $\tau \cdot (2^{16} - 1)$, so maintaining $\tau \leq 2^{16} - 1$ keeps every limb below 2^{32} and therefore inside the baby-step/giant-step recovery domain. The ledger maintains the bound through the following rules, referred to below as the *counter invariant*.

Counter invariant.

- A fresh empty aggregate has $\tau = 0$; a freshly verified well-formed encryption has $\tau = 1$. Every operation that installs a fresh range-proven active balance (registration, transfer, unwrap, update balance) leaves $\tau_{\text{act}} = 1$, and each incoming credit increments the receiver's τ_{pnd} by one. Merge adds τ_{pnd} to τ_{act} , plus one if the public accumulator is nonzero, and clears the pending state.
- A credit to receiver R is admitted only if $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$: one slot for the credit and one reserved for the eventual merge of the public accumulator. The reserved slot is claimed by the first wrap since the last merge, so while the accumulator is nonzero further wraps consume no new slot, whereas a wrap onto an empty accumulator is subject to the same condition.
- Merge and update balance (Section 3.2) are independent: either may be invoked on its own and in either order. Update balance reads and rewrites only the active balance and never raises τ_{act} , so it preserves or decreases the joint counter $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R$ and the reserved-slot accounting remains sound no matter when, or with what pending state, it is called.

The separation between active and pending state is essential. Incoming transfers do not immediately alter the recipient's active balance. Instead, they are accumulated and later merged in one homomorphic step. This design allows the sender's transfer proof to remain local to the sender.

3.2 Confidential Transfers

Public-to-private conversion. Suppose a user wishes to *wrap* a public coin of value v into confidential balance for some registered receiver R . The ledger deposits the public coin into the shared pool associated with the confidential asset and increments the receiver's pending public accumulator by v . No encryption or proof accompanies this step: the same public value v that enters the pool is recorded in the accumulator, so the ledger itself enforces equality between the public and private representations. The resulting value is not yet part of the receiver's active confidential balance; it enters the active state only after a later merge step.

Private transfer. A private transfer moves confidential value from a single sender S with public key pk_S and active encrypted balance B_S to a list of registered receivers $R_1, \dots, R_K$ with public keys pk_{R_k} and per-receiver amounts a_k . The single-receiver transfer is the special case $K = 1$.

The sender first decrypts B_S locally, computes the remaining balance $b'_S = b_S - \sum_{k=1}^K a_k$, and re-encrypts it as a well-formed four-limb ciphertext $B'_S = \text{EA}(\text{pk}_S, b'_S)$. It then samples a fresh ephemeral scalar e , publishes the point $P = eG$, and forms the sender-side Diffie–Hellman secret $Z_S = e \text{pk}_S = \text{sk}_S P$. For each receiver R_k , the sender derives its per-limb blindings $\rho_{k,i} = \text{KDF}(Z_S, \cdot)$ from Z_S using a key-derivation hash KDF (a domain-separated hash, modeled as a random oracle in the security analysis) and encrypts the corresponding transfer amount as $A_{R_k} = \text{EA}(\text{pk}_{R_k}, a_k; \rho_k)$. Deriving the blindings from Z_S lets the sender, and any auditor that recovers sk_S , reconstruct every

$\rho_{k,i}$ from $Z_S = \text{sk}_S P$ and read each a_k off the on-chain receiver commitments, so no separate sender-side amount ciphertext need be posted. Because the twisted-ElGamal commitment is independent of the public key (Section 8), the sender’s aggregate debit reuses the very same commitments: writing $r_S = \sum_{k,i} 2^{16i} \rho_{k,i}$ for the collapsed blinding, the aggregate sender-side ciphertext is

$$\hat{A}_S = (\hat{A}_S.C, \hat{A}_S.D), \quad \hat{A}_S.C = \sum_{k=1}^K \text{Collapse}(A_{R_k}).C, \quad \hat{A}_S.D = r_S \text{pk}_S,$$

whose commitment the ledger reconstructs from the receiver commitments; only the single aggregate handle $\hat{A}_S.D$ is posted.

Callers may attach an optional opaque memo to wrap and transfer operations, recorded verbatim in the public log; we omit memos from the cryptographic description below. The sender submits B'_S , the ephemeral point P , the receiver ciphertexts $(A_{R_k})_{k=1}^K$, the aggregate handle $\hat{A}_S.D$, and proofs as transaction arguments. First, the sender attaches well-formedness material for B'_S and for every A_{R_k} : one batched range proof covers all $4(K+1)$ limb commitments (following the implementation), and the ElGamal consistency proofs are witness-folded per key (Appendix B). A single folded proof γ_S under pk_S covers the four limbs of B'_S together with the aggregate $\hat{A}_S = (\hat{A}_S.C, \hat{A}_S.D)$, certifying in particular that $\hat{A}_S$ is a well-formed Twisted ElGamal encryption under pk_S of the total $\sum_k a_k$ with some aggregate blinding the sender knows; one folded proof γ_{R_k} under each pk_{R_k} covers that receiver’s four limbs. Second, the sender includes a single DDH proof that the old balance equals the new balance plus the total transferred amount. Writing

$$\Delta = \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S),$$

the intended relation is that Δ encrypts zero under pk_S . This is certified by a DDH proof showing that the pair $(\Delta.C, \Delta.D)$ is consistent with secret key sk_S relative to (G, pk_S) .

Per-pair binding “the sender debited a'_k on behalf of R_k ” is *not* enforced; the ledger only ties the totals. The ledger reconstructs the aggregate commitment

$$\hat{A}_S.C = \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$$

directly from the receiver commitments, which together with the range proofs over each A_{R_k} (pinning every a_k in $[0, 2^{16})$ per limb, and so $\sum_k a_k$ in $\mathbb{Z}$ below the group order) and the sender-side folded consistency proof γ_S forces the aggregate sender-side debit to equal $\sum_k a_k$ exactly.

Private-to-public conversion. To withdraw, or *unwrap*, a public amount v from a confidential balance, a holder U with public key pk_U decrypts the current active balance, subtracts v locally, and re-encrypts the resulting amount as a well-formed ciphertext $B'_U = \text{EA}(\text{pk}_U, b'_U)$. The holder then considers the difference

$$\Delta = \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U).$$

If the claimed withdrawal is correct, then Δ encrypts zero under pk_U . The holder therefore submits B'_U , the public value v , a range proof and a witness-folded ElGamal consistency proof for B'_U (as in Definition 1), and a DDH proof certifying that Δ is a zero encryption under pk_U . Once verified, the ledger updates the active balance to B'_U and releases v units of the public coin from the shared pool.

Merging pending deposits and restoring well-formedness. Pending deposits are absorbed into the active balance by two ledger operations, *merge* and *update balance*, whose counter accounting is governed by the invariant of Section 3.1. Merge folds the pending balance into the active one using homomorphic additions of encryptions. This preserves the plaintext balance, but not the property that every limb lies in $[0, 2^{16})$: repeated homomorphic addition may produce limbs whose plaintexts exceed that range even though the overall balance remains decryptable. A holder may invoke merge multiple times before updating the balance, each merge advancing the bound counter τ ; the ledger aborts a pending deposit that would exhaust the remaining slack.

Update balance restores limb well-formedness, and with it the slack that admits further deposits. The holder decrypts the active ciphertext, re-decomposes the plaintext into fresh 16-bit limbs, and re-encrypts under the same public key to obtain a new well-formed encrypted amount B'_U . The holder then proves that B'_U represents exactly the same value as the previous active balance by forming

$$\Delta = \text{Collapse}(B'_U) - \text{Collapse}(B_U)$$

and certifying, again with a DDH proof relative to (G, pk_U) , that Δ encrypts zero. The holder submits B'_U together with a range proof and a folded consistency proof for those fresh limbs, and the ledger verifies both well-formedness and equality and replaces the active balance by B'_U , resetting τ_{act} to 1.

In practice, the holder rarely pays for a balance update as a standalone operation. Every transfer and unwrap already supplies a fresh well-formed new balance with its own range proof and folded consistency proof, so a merge followed by a spend in the same transaction lets the spend's proofs double as the balance update, saving one bulletproof aggregate and one consistency proof. The optimistic spend variants of the implementation also compose naturally with this pattern: if a pending deposit lands between the holder's view of state and execution and the post-merge balance proof fails, the merge still commits and the spend leg short-circuits with a failure event instead of aborting the transaction.

3.3 Auditing

Design choice. Two designs for exposing confidential state to auditors were considered. The first encrypts the payload of every confidential transfer directly to each of the n auditors, attaching n additional ciphertexts and n consistency proofs to every transaction. The second encrypts each account's secret viewing key once, at registration or key-rotation time, to every auditor in the list, so that auditors acquire a persistent decryption capability over the account. The protocol adopts the second, per-account design for two reasons. First, per-transaction encryption would linearly inflate the size of the system's most frequent operation, whereas the per-account approach adds a fixed, one-off cost at registration and no overhead on subsequent transfers, unwraps, or merges. Second, the per-account design makes auditor monitoring stateless: a newly on-boarded auditor can, with its decryption key alone, recover the secret key of any registered account and independently decrypt all current and historical confidential state directly from the ledger, without a replay of the transaction graph or access to old auditor keys. The remaining drawback, that a leaked auditor key retains decryption power against existing encapsulations, is mitigated by a mechanism described below.

Account registration with key encapsulation. To register a confidential account under a fresh key pair $(\text{sk}_U, \text{pk}_U)$, the holder reads the current auditor list $\mathcal{J} = (\text{pk}_{A_1}, \dots, \text{pk}_{A_n})$ from the ledger and decomposes sk_U into eight 32-bit limbs $(u_0, \dots, u_7)$, so that $\text{sk}_U = \sum_{i=0}^7 2^{32i} u_i$. The

holder encrypts each limb u_i as a multi-recipient Twisted ElGamal ciphertext under $\mathcal{J}$ with fresh blinding r_i , yielding

$$(C_i, (D_{ij})_{j=1}^n), \quad C_i = r_i G + u_i H, \quad D_{ij} = r_i \text{pk}_{A_j}.$$

The holder produces a bit-width-32 range proof π_{range}^{32} over the eight limb commitments and a key-consistency proof π_{KC} binding the encrypted limbs to pk_U . The encapsulation

$$\mathcal{E} = ((C_i, (D_{ij})_j)_{i=0}^7, \pi_{\text{range}}^{32}, \pi_{\text{KC}})$$

is submitted together with an initialization of the account's active and pending state. The ledger verifies $\text{pk}_U \neq \mathcal{O}$, verifies π_{range}^{32} against $(C_0, \dots, C_7)$ and verifies π_{KC} against pk_U , $\mathcal{J}$, and the limb ciphertexts; it stores $\mathcal{E}$ against the account and initializes the active balance to $\text{EA}(\text{pk}_U, 0)$. The formal procedure appears as Algorithm 1.

Key rotation (Algorithm 8) re-keys the active balance in place: the holder keeps the existing limb commitments, replaces each decryption handle with one under the new nonzero public key, and proves with a batched DDH proof Π_{rekey} that the new handle is the old one using the witness $w = \text{sk}'_U \text{sk}_U^{-1}$, while a fresh encapsulation rebinds the auditor list to the new key.

Auditor list management. The issuer may insert or remove entries from $\mathcal{J}$ at any time, as recorded in Algorithm 2. By default, such changes apply only to future registrations and key rotations; existing encapsulations remain valid and continue to grant decryption capability to whichever auditors they were sealed against. If the issuer determines that the previous list must no longer hold decryption capability, for example after a suspected key leakage, the issuer may additionally signal on the ledger a *recommended update*. In that case wallets are expected to refresh the user key and encapsulate it for the new set $\mathcal{J}$ (a step the wallet may bundle into the next user-initiated transaction).

Auditor decryption flow. Auditing is an entirely off-chain, read-only operation. To audit account U , auditor A_j reads the stored encapsulation $\mathcal{E}$ from the ledger, projects each multi-recipient limb ciphertext $(C_i, (D_{ij'})_{j'})$ to its own slot to obtain the single-recipient ciphertexts $((C_i, D_{ij}))_{i=0}^7$, and decrypts each limb with sk_{A_j} using the same baby-step/giant-step procedure used for balance limbs. The auditor reassembles the limbs into sk_U and checks that $\text{sk}_U G = \text{pk}_U$; since π_{KC} was verified on-chain, this check succeeds for any honestly recorded encapsulation. With sk_U in hand, the auditor decrypts the account's current active balance, its pending encrypted accumulator, and the receiver side of any confidential transfer the account has received (by decrypting that transfer's receiver-side amount ciphertext A_R under $\text{sk}_U = \text{sk}_R$). For confidential transfers the account has *sent*, the auditor reads the transfer's ephemeral point P off the ledger and recomputes the sender-side secret $Z_S = \text{sk}_U P$; when the sender followed the protocol, it re-derives every per-limb blinding $\rho_{k,i} = \text{KDF}(Z_S, \cdot)$ from Z_S and reads each per-leg amount a_k directly off the on-chain receiver commitment A_{R_k} (subtracting $\rho_{k,i} G$ from each limb commitment and solving the bounded discrete log), recovering an exact per-pair amount for every k , not merely the account-level total. The account-level debit is in any case fixed by the drop in the sender's balance, which the auditor decrypts directly, so account totals are always recoverable. Since the encapsulation is created once per account, the only per-transaction cost of auditing is the single ephemeral group element P , independent of the number of receivers.

4 Security Discussion

This section gives an informal account of the protocol’s integrity and privacy guarantees; Section 6 establishes them formally as UC-realization of $\mathcal{F}_{\text{ConfTx}}$.

Integrity of balances. The principal safety property is conservation of value between the encrypted state and the public backing pool, and each operation preserves it by construction. Wrap credits a recipient’s pending state with exactly the public value locked into the pool. A confidential transfer is accepted only when the sender proves that the old active balance equals the new active balance plus the transferred amount, and that the sum of the amounts credited across the receivers equals the amount debited from the sender (Algorithm 5). Unwrap requires a proof that the decrease in encrypted balance matches the public value released from the pool (Algorithm 6). Merge and balance update compose homomorphic addition with a re-encryption proved equal under Π_{DDH} , and are therefore value preserving (Algorithm 7). Key rotation reuses the existing commitments and only re-keys the decryption handles, so it changes no plaintext (Algorithm 8).

Non-negativity. Every balance eligible to become active after transfer, unwrap, or balance update is re-encoded as four limbs, each proved to lie in $[0, 2^{16})$, so at the moment of verification the encoded plaintext lies in $[0, 2^{64})$. The bound counter $\tau < 2^{16}$ (Section 3.1) keeps accumulated limbs decryptable between verifications. Together with the balance relation checked at each state transition, no polynomial-time adversary can make an account spend more than its available value without breaking proof soundness or the hardness assumptions underlying the encryption scheme.

Theorem 1 (Informal integrity statement). *Assuming the discrete-log problem is hard in $\mathbb{G}$ and the proof systems Π_{DDH} , Π_{EG} , Π_{range} , and Π_{rekey} are sound, no polynomial-time adversary can cause the sum of all confidential balances to exceed the amount of public coin held in the corresponding backing pool, provided the issuer does not invoke the privileged write paths described below.*

Spending requires owner authentication, not merely sk_U . A transfer, unwrap, or balance update is accepted only if two independent conditions hold: the ledger authenticates the caller as the account’s owner, and the submitted balance, range, and consistency proofs verify. The secret key sk_U is relevant only to the second condition. It allows its holder to decrypt the account’s ciphertexts and produce the Π_{DDH} balance proof, but it carries no authority to invoke an operation on the account: each state transition is tied to the holder’s identity by the ledger; the implementation requires that the transaction sender owns the account. Holding sk_U is therefore necessary but not sufficient to spend. In particular, an auditor that recovers sk_U through the key encapsulation, or any other party that comes to hold it, gains only a read capability: it can decrypt every balance and transfer amount of the account, but it cannot move funds, since it cannot authenticate as the owner.

Issuer-controlled escape hatches. The integrity statement is conditioned on the issuer not exercising a few privileged operations that the implementation exposes for compliance and emergency response and that intentionally bypass conservation: a global pause that disables transfers and unwraps; a per-account deny list that prevents an address from sending or receiving; and a direct overwrite of a registered account’s encrypted balance, used to emulate burn or seize. The first two are halting operations and preserve conservation between encrypted state and the backing pool. The overwrite does not: it can leave the sum of confidential balances out of step with the pool, so the issuer must use it only when reconciliation is restored out of band. Holders accept this trust assumption when they register against an issuer’s confidential token.

Privacy of transactions. Privacy derives from two sources. Twisted ElGamal is semantically secure under DDH in $\mathbb{G}$ [CMTA20, ElG85], and its ciphertext component is a perfectly hiding Pedersen commitment; the multi-recipient variant used for key encapsulation retains semantic security under the same assumption, since each decryption handle is an independent DDH tuple under the corresponding recipient key. A transfer derives its per-limb blindings deterministically from the sender-side Diffie–Hellman secret $Z_S = \text{sk}_S P$ through a key-derivation hash KDF, modeled as a random oracle (Section 3.2); under DDH the value Z_S is pseudorandom to any observer holding neither sk_S nor the ephemeral scalar, so that observer cannot query the oracle at Z_S and the derived blindings are uniformly random in its view, leaving the semantic security above unchanged. The correctness proofs are zero knowledge [FS87, BBB⁺18], so they reveal nothing about the encrypted amount, the blinding randomness, the encapsulated secret key, or the holder’s long-term secret beyond the validity of the claimed relation. An observer of a private transfer therefore learns only that the sender’s encrypted balance was updated consistently and that each receiver obtained a well-formed encrypted amount equal to the sender’s debit; the amounts themselves remain hidden.

The privacy claim is intentionally limited in two ways. Wrap and unwrap amounts are public, since they move value across the boundary between public and private representations, and the transaction graph is visible because the ledger records which accounts interact; the protocol thus hides confidential balances and transfer amounts but not participation patterns or boundary crossings. Second, any party holding the decryption key of an auditor currently registered against an account learns that account’s secret viewing key, and hence every confidential balance and transfer amount associated with it. This is a read capability only, since sk_U does not authorize spending (as noted above). The key encapsulation grants no capability beyond that secret key, and leaks nothing about accounts whose encapsulation does not include the auditor.

Theorem 2 (Informal privacy statement). *Assuming DDH in $\mathbb{G}$, modeling the transfer key-derivation hash KDF as a random oracle, and assuming the zero-knowledge property of Π_{DDH} , Π_{EG} , Π_{range} , Π_{KC} , and Π_{rekey} , a polynomial-time observer of the ledger who does not hold the decryption key of any currently-registered auditor learns no information about confidential transfer amounts or active confidential balances beyond what follows from the public wrap and unwrap values together with the visible transaction graph. A polynomial-time observer holding sk_{A_j} for an auditor A_j additionally learns the secret viewing key, and hence every confidential balance and transfer amount, of every account whose stored encapsulation includes pk_{A_j} .*

Confidentiality and corruption. We call the privacy guarantee against a polynomial-time adversary that holds no auditor decryption key *confidentiality* and treat it as the primary privacy notion. The general case, in which a static adversary also holds sk_{A_j} for some auditor A_j , reduces to it: the encapsulation makes sk_{A_j} deterministically yield the secret viewing key of every account whose stored encapsulation includes pk_{A_j} , and the auditor’s view is a deterministic function of the public ledger, so those accounts fold into the corruption set and confidentiality applies to the remaining honest accounts.

The auditor list can change over time: an issuer update (Algorithm 2) may add a corrupt auditor key. This does not affect existing accounts, whose encapsulation is fixed at registration or key rotation. A corrupt auditor sees an account only if that account later re-encapsulates under a list containing the corrupt key, so adaptive auditor corruption reduces to the static case at each user’s last re-encapsulation.

See Section 7 for a comparison with prior security models.

Algorithm 1 Account registration with verifiable key encapsulation

Require: Fresh holder key pair $(\text{sk}_U, \text{pk}_U)$ with $\text{pk}_U = \text{sk}_U G$; current auditor list $\mathcal{J} = (\text{pk}_{A_1}, \dots, \text{pk}_{A_n})$.

Holder (off-chain)

- 1: decompose sk_U into 32-bit limbs $(u_0, \dots, u_7)$ with $\text{sk}_U = \sum_{i=0}^7 2^{32i} u_i$
- 2: **for** $i = 0$ **to** 7 **do**
- 3: $r_i \xleftarrow{\$} \mathbb{Z}_q$; $E_i = (C_i, (D_{ij})_j)$ with $C_i = r_i G + u_i H$, $D_{ij} = r_i \text{pk}_{A_j}$
- 4: **end for**
- 5: $\pi_{\text{range}}^{32} \leftarrow \Pi_{\text{range}}^{32}((u_i, r_i)_i; (C_0, \dots, C_7))$ $\triangleright$ ensures each key limb is a genuine 32-bit value
- 6: $\pi_{\text{KC}} \leftarrow \Pi_{\text{KC}}((u_i, r_i)_i; \text{pk}_U, \mathcal{J}, (E_i)_i)$ $\triangleright$ ensures every auditor ciphertext encrypts the same key bits, and they match pk_U , so any auditor can recover sk_U
- 7: $\mathcal{E} \leftarrow ((E_i)_{i=0}^7, \pi_{\text{range}}^{32}, \pi_{\text{KC}})$
- 8: **submit** $(\text{pk}_U, \mathcal{E})$

Ledger (on-chain)

- 9: **assert** $\text{pk}_U \neq \mathcal{O}$
 - 10: **assert** $\text{Verify}_{\text{range}}^{32}(\pi_{\text{range}}^{32}; (C_0, \dots, C_7))$
 - 11: **assert** $\text{Verify}_{\text{KC}}(\pi_{\text{KC}}; \text{pk}_U, \mathcal{J}, (E_i)_i)$
 - 12: store the encapsulation ciphertexts $(E_i)_{i=0}^7$ against the account
 - 13: $B_U \leftarrow \text{EA}(\text{pk}_U, 0)$ with bound counter $\tau_{\text{act}} \leftarrow 1$ ($\text{EA}(\text{pk}_U, 0)$ is a well-formed encryption);
 $B_{\text{pnd}}^U \leftarrow \mathbf{0}$, $\tau_{\text{pnd}} \leftarrow 0$ (encrypted pending balance); $b_{\text{pub}}^U \leftarrow 0$ (public pending balance)
- Ensure:** Key rotation reuses this encapsulation step under the new key; see Algorithm 8.
-

5 Formal Description

The following pseudocode uses the primitives from Section 2: **Enc**, **Dec**, **EA**, **Collapse**, **TrivEnc**, Π_{DDH} , Π_{EG} , $\Pi_{\text{EG}}^{\otimes}$, Π_{range} , Π_{KC} , and **Verify**. The map $\text{TrivEnc}_{\text{limbs}}$ splits a u64 into four 16-bit limbs and applies **TrivEnc** to each. For a Twisted ElGamal ciphertext $E = (C, D)$ we write $E.C$ and $E.D$ for its components. For a multi-recipient ciphertext $E = (C, (D_j)_j)$ we write $E.C$ and $E.D_j$ accordingly. Write a four-limb ciphertext as $B = (B_{[0]}, \dots, B_{[3]})$. When the holder forms $B = \text{EA}(\text{pk}, v)$, let (v_i, ρ_i) denote the openings with $B_{[i]} = \text{Enc}(\text{pk}, v_i; \rho_i)$; when several ciphertexts are in play, the blindings carry the account subscript $(\rho_{S,i}, \rho_{U,i})$ and a prime carries over from a ciphertext to its limb values, so the openings of B'_S are $(v'_i, \rho_{S,i})$. A superscript on Π_{range} , π_{range} , or $\text{Verify}_{\text{range}}$ records the bit-width of the proven range: 16 for balance and amount limbs, 32 for key-encapsulation limbs. Lines marked **Holder** run off-chain: the holder uses witnesses to *compute* proof objects (notation $\pi \leftarrow \Pi_{\dots}$). Lines marked **Ledger** run on-chain: the verifier only *checks* submitted ciphertexts and proofs (notation $\text{Verify}(\dots)$ or **assert**). Proofs are carried as transaction inputs; they are not recomputed by the ledger. A witness-folded consistency proof $\Pi_{\text{EG}}^{\otimes}((m_j, r_j)_j; \text{pk}, (E_j)_j)$ certifies all listed same-key ciphertexts at once at the cost of a single Π_{EG} ; $\text{Verify}_{\text{EG}}^{\otimes}(\gamma; \text{pk}, (E_j)_j)$ denotes its verifier (Appendix B).

Algorithms 1–8 give the protocol; confidential transfer is split across Algorithms 4 and 5 (off-chain holder vs. on-chain ledger).

Algorithm 2 Auditor list update (issuer)

Require: Issuer authority over asset T ; new auditor list $\mathcal{J}' = (\text{pk}'_{A_1}, \dots, \text{pk}'_{A_{n'}})$.

Ledger (on-chain, authorized)

1: $\mathcal{J} \leftarrow \mathcal{J}'$

Ensure: Subsequent registrations and key rotations (Algorithm 1) must use $\mathcal{J}$. Issuer may also signal to wallets that a key rotation is recommended.

Algorithm 3 Public-to-private conversion (wrap)

Require: Registered recipient R ; $v \in \text{u64}$ (value of the public coin consumed); $b_{\text{pub}}^R > 0$ or $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$ (the reserved public-merge slot of Section 3.1 is claimed by the first wrap since the last merge; later wraps reuse it).

1: $\text{pool} \leftarrow \text{pool} + v \quad \triangleright \text{public coin locked into the shared backing } \textit{Balance} \text{ for the asset}$

2: $b_{\text{pub}}^R \leftarrow b_{\text{pub}}^R + v \quad \triangleright \text{receiver's pending public accumulator; active ciphertext } B_R \text{ unchanged here}$

Ensure: No zero-knowledge proof; first nonzero active ciphertext appears at merge (Algorithm 7).

Algorithm 4 Private transfer: holder (off-chain)

Require: Sender keys $(\text{sk}_S, \text{pk}_S)$; receiver keys $(\text{pk}_{R_k})_{k=1}^K$; on-chain active balance B_S ; amounts $(a_k)_{k=1}^K$.

- 1: $b_S \leftarrow \text{Dec}(\text{sk}_S, B_S)$
- 2: $b'_S \leftarrow b_S - \sum_{k=1}^K a_k$; $B'_S \leftarrow \text{EA}(\text{pk}_S, b'_S)$
- 3: $e \xleftarrow{\$} \mathbb{Z}_q$; $P \leftarrow eG$; $Z_S \leftarrow e \text{pk}_S \triangleright$ *fresh ephemeral; sender-side DH secret $Z_S = \text{sk}_S P$*
- 4: **for** $k = 1$ **to** K **do**
- 5: $\rho_{k,i} \leftarrow \text{KDF}(Z_S, \text{"amt"} \| k \| i)$ for $i \in \{0, \dots, 3\}$
 $A_{R_k} \leftarrow \text{EA}(\text{pk}_{R_k}, a_k; \rho_k) \triangleright$ *per-limb blindings derived from the DH secret Z_S , so the sender's auditor recovers a_k from A_{R_k}*
- 6: **end for**
- 7: $r_S \leftarrow \sum_{k,i} 2^{16i} \rho_{k,i}$; $\hat{A}_S.C \leftarrow \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$; $\hat{A}_S.D \leftarrow r_S \text{pk}_S \triangleright$ *aggregate sender-side ciphertext: commitment reused from the receiver commitments, only its handle $\hat{A}_S.D$ is posted*
- 8: $\Delta \leftarrow \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$
 $\pi_{\text{bal}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_S; G, \Delta.C, \text{pk}_S, \Delta.D) \triangleright$ *ensures Δ encrypts zero: what leaves the sender equals the total credited, so no value is minted*
- 9: $\mathcal{L}_{\text{rng}} \leftarrow (B'_{S,[0]}.C, \dots, B'_{S,[3]}.C)$
 $\| (A_{R_k,[0]}.C, \dots, A_{R_k,[3]}.C)_{k=1}^K \triangleright 4(K+1)$ *limb commitments for one batched range proof; the aggregate $\hat{A}_S$ reuses the receiver limb commitments and needs no separate entries*
- 10: $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}}) \triangleright$ *ensures every new-balance and receiver-amount limb is in $[0, 2^{16})$, ruling out a negative residual or limb overflow that would fabricate value*
- 11: $\gamma_S \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{S,i})_{i=0}^3, (\sum_k a_k, r_S); \text{pk}_S, (B'_{S,[i]})_{i=0}^3, \hat{A}_S) \triangleright$ *one witness-folded proof for all five sender-keyed ciphertexts: each new-balance limb is well-formed under pk_S (the sender can still decrypt its own balance) and the aggregate handle $\hat{A}_S.D$ is pinned to pk_S , not an arbitrary point*
- 12: **for** $k = 1$ **to** K **do**
- 13: $\gamma_{R_k} \leftarrow \Pi_{\text{EG}}^{\otimes}((a_{k,i}, \rho_{k,i})_{i=0}^3; \text{pk}_{R_k}, (A_{R_k,[i]})_{i=0}^3) \triangleright$ *one folded proof per receiver: every amount limb is well-formed under pk_{R_k} , so the recipient can decrypt the funds sent*
- 14: **end for**
- 15: **submit** $(B'_S, P, (A_{R_k})_{k=1}^K, \hat{A}_S.D, \pi_{\text{range}}^{16}, \gamma_S, (\gamma_{R_k})_{k=1}^K, \pi_{\text{bal}}) \triangleright$ *only P and the single aggregate handle $\hat{A}_S.D$ are posted; $K+1$ consistency proofs in total, one per key, independent of the limb count*

Algorithm 5 Private transfer: ledger (on-chain)

Require: Transaction inputs from Algorithm 4; on-chain state: the asset is not globally paused; sender account S with active balance B_S and public key pk_S is registered, not denied, and not frozen; each receiver R_k with public key pk_{R_k} , active counter $\tau_{\text{act}}^{R_k}$, and pending encrypted accumulator $B_{\text{pnd}}^{R_k}$ with counter $\tau_{\text{pnd}}^{R_k}$ is registered, not denied, not frozen, accepts encrypted deposits, and satisfies $\tau_{\text{act}}^{R_k} + \tau_{\text{pnd}}^{R_k} + 2 < 2^{16}$ (one slot for the credit, one reserved for the eventual public-pending merge).

- 1: **assert** $K \geq 1$ $\triangleright$ *nonempty batch*
 $\triangleright$ *Well-formedness of new balance, per-receiver amounts, and aggregate debit*
 - 2: **assert** $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
 - 3: $\hat{A}_S.C \leftarrow \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$; $\hat{A}_S.D \leftarrow$ submitted aggregate handle $\triangleright$ *commitment reconstructed on-chain from the receiver commitments*
 - 4: **assert** $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_S; \text{pk}_S, (B'_{S,[i]})_{i=0}^3, \hat{A}_S)$ $\triangleright$ *one folded check under pk_S : the four new-balance limbs and the aggregate sender-side debit are consistent encryptions*
 - 5: **for** $k = 1$ **to** K **do**
 - 6: **assert** $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_{R_k}; \text{pk}_{R_k}, (A_{R_k,[i]})_{i=0}^3)$ $\triangleright$ *one folded check per receiver key*
 - 7: **end for**
 $\triangleright$ *Balance relation*
 - 8: $\Delta \leftarrow \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$
 - 9: **assert** $\text{Verify}_{\text{DDH}}(\pi_{\text{bal}}; G, \Delta.C, \text{pk}_S, \Delta.D)$ $\triangleright$ Δ *encrypts zero under pk_S : what leaves the sender equals the total credited*
 - 10: $B_S \leftarrow B'_S$; $\tau_{\text{act}}^S \leftarrow 1$ $\triangleright$ B'_S *is freshly range-proven, so the sender's bound counter resets*
 - 11: **for** $k = 1$ **to** K **do**
 - 12: $B_{\text{pnd}}^{R_k} \leftarrow B_{\text{pnd}}^{R_k} + A_{R_k}$; $\tau_{\text{pnd}}^{R_k} += 1$ $\triangleright$ *limb-wise homomorphic add into R_k 's pending accumulator*
 - 13: **emit** $\text{TransferEvent}(S, R_k, A_{R_k})$ $\triangleright$ P *is recorded once per transfer*
 - 14: **end for**
-

Algorithm 6 Private-to-public conversion (unwrap)

Require: Holder keys $(\text{sk}_U, \text{pk}_U)$; on-chain active balance B_U ; public withdrawal $v \in \text{u64}$.

Holder (off-chain)

- 1: $b_U \leftarrow \text{Dec}(\text{sk}_U, B_U)$; $b'_U \leftarrow b_U - v$; $B'_U \leftarrow \text{EA}(\text{pk}_U, b'_U)$
- 2: $\Delta \leftarrow \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U)$
- 3: $\pi_{\text{bal}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_U; G, \Delta.C, \text{pk}_U, \Delta.D) \triangleright \text{ensures } \Delta \text{ encrypts zero: the public withdrawal } v \text{ exactly matches the drop in the encrypted balance}$
- 4: $\mathcal{L}_{\text{rng}} \leftarrow (B'_{U,[0]}.C, \dots, B'_{U,[3]}.C)$
- 5: $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}}) \triangleright \text{ensures the residual balance limbs are non-negative, so the holder cannot withdraw more than it holds}$
- 6: $\gamma_U \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{U,i})_{i=0}^3; \text{pk}_U, (B'_{U,[i]})_{i=0}^3) \triangleright \text{one witness-folded proof: every residual-balance limb is well-formed under } \text{pk}_U, \text{ keeping the balance decryptable}$
- 7: **submit** $(B'_U, v, \pi_{\text{range}}^{16}, \gamma_U, \pi_{\text{bal}})$

Ledger (on-chain)

- 8: **assert** $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
 - 9: **assert** $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_U; \text{pk}_U, (B'_{U,[i]})_{i=0}^3) \triangleright \text{one folded check for all four residual-balance limbs}$
 - 10: $\Delta \leftarrow \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U)$
 - 11: **assert** $\text{Verify}_{\text{DDH}}(\pi_{\text{bal}}; G, \Delta.C, \text{pk}_U, \Delta.D) \triangleright \Delta \text{ encrypts zero under } \text{pk}_U: \text{the public withdrawal } v \text{ matches the balance drop}$
 - 12: $B_U \leftarrow B'_U$; $\tau_{\text{act}} \leftarrow 1$; $\text{pool} \leftarrow \text{pool} - v$; **return** public coin of value v
-

Algorithm 7 Merge pending deposits; update balance (two independent operations)

Require: Account state: active balance B_U with bound counter τ_{act} , pending encrypted B_{pnd}^U (with τ_{pnd}), pending public b_{pub}^U ; holder secret sk_U .

- ▷ *MERGE (ledger): an independent operation that folds pending into active*
- 1: $B_U \leftarrow B_U + B_{\text{pnd}}^U + \text{TrivEnc}_{\text{limbs}}(b_{\text{pub}}^U)$ ▷ *homomorphic add pending encrypted + trivial limb encoding of pending public*
 - 2: $\tau_{\text{act}} \leftarrow \tau_{\text{act}} + \tau_{\text{pnd}} + \mathbf{1}[b_{\text{pub}}^U > 0]$ ▷ *see Section 3.1 for the $\tau < 2^{16}$ invariant; the public accumulator consumes a slot only when nonzero*
 - 3: $B_{\text{pnd}}^U \leftarrow \mathbf{0}, \tau_{\text{pnd}} \leftarrow 0$ ▷ *reset pending encrypted; zero four-limb ciphertext*
 - 4: $b_{\text{pub}}^U \leftarrow 0$

▷ *UPDATE BALANCE (ledger + holder): an independent operation, callable on its own*
 ▷ *it reads and rewrites only B_U ; it does not require $\tau_{\text{pnd}} = 0$ and never touches pending*

Holder (off-chain)

- 5: $b_U \leftarrow \text{Dec}(\text{sk}_U, B_U); B'_U \leftarrow \text{EA}(\text{pk}_U, b_U)$ ▷ *fresh limbs under pk_U*
- 6: $\Delta \leftarrow \text{Collapse}(B'_U) - \text{Collapse}(B_U)$
- 7: $\pi_{\text{eq}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_U; G, \Delta.C, \text{pk}_U, \Delta.D)$ ▷ *ensures Δ encrypts zero: the freshly re-encrypted balance holds the same value as the merged one*
- 8: $\mathcal{L}_{\text{rng}} \leftarrow (B'_{U,[0]}.C, \dots, B'_{U,[3]}.C)$
- 9: $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}})$ ▷ *ensures the re-encoded limbs are back in $[0, 2^{16})$, restoring the limb bound after homomorphic accumulation*
- 10: $\gamma_U \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{U,i})_{i=0}^3; \text{pk}_U, (B'_{U,[i]}_{i=0}^3))$ ▷ *one witness-folded proof: every re-encoded limb is well-formed under pk_U , so the refreshed balance stays decryptable*
- 11: **submit** $(B'_U, \pi_{\text{range}}^{16}, \gamma_U, \pi_{\text{eq}})$

Ledger (on-chain)

- 12: **assert** $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
 - 13: **assert** $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_U; \text{pk}_U, (B'_{U,[i]}_{i=0}^3))$ ▷ *one folded check for all four re-encoded limbs*
 - 14: $\Delta \leftarrow \text{Collapse}(B'_U) - \text{Collapse}(B_U)$
 - 15: **assert** $\text{Verify}_{\text{DDH}}(\pi_{\text{eq}}; G, \Delta.C, \text{pk}_U, \Delta.D)$ ▷ *Δ encrypts zero under pk_U : the refreshed balance equals the merged one*
 - 16: $B_U \leftarrow B'_U; \tau_{\text{act}} \leftarrow 1$
-

Algorithm 8 Account key rotation

Require: Current key $(\text{sk}_U, \text{pk}_U)$ with $\text{pk}_U \neq \mathcal{O}$; fresh key $(\text{sk}'_U, \text{pk}'_U)$ with $\text{pk}'_U = \text{sk}'_U G$; active balance B_U with limbs $B_{U,[i]} = \text{Enc}(\text{pk}_U, v_i; \rho_{U,i})$ and merge counter τ_{act} (carried over unchanged); auditor list $\mathcal{J}$; no pending *encrypted* deposits ($\tau_{\text{pnd}} = 0$). The public accumulator b_{pub}^U need not be zero.

Holder (off-chain)

- 1: $w \leftarrow \text{sk}'_U \text{sk}_U^{-1}$
- 2: **for** $i = 0$ **to** 3 **do**
- 3: $D'_i \leftarrow w B_{U,[i]}.D \triangleright$ reuse the commitment $B_{U,[i]}.C$ and re-key only the handle.
- 4: **end for**
- 5: $\pi_{\text{rekey}} \leftarrow \Pi_{\text{rekey}}(w; \text{pk}_U, \text{pk}'_U, (B_{U,[i]}.D, D'_i)_{i=0}^3) \triangleright$ one batched DDH: the quotient w maps $\text{pk}_U \mapsto \text{pk}'_U$ and every old handle to its re-keyed counterpart
- 6: $\mathcal{E}' \leftarrow$ fresh encapsulation of sk'_U under $\mathcal{J}$, with π_{range}^{32} and π_{KC} as in Algorithm 1
- 7: **submit** $(\text{pk}'_U, (D'_i)_{i=0}^3, \pi_{\text{rekey}}, \mathcal{E}')$ $\triangleright$ only the re-keyed handles posted; the ledger already holds the reused commitments $B_{U,[i]}.C$

Ledger (on-chain)

- 8: **assert** $\text{pk}'_U \neq \mathcal{O}$
 - 9: **assert** $\tau_{\text{pnd}} = 0 \triangleright$ no pending encrypted deposits.
 - 10: **assert** $\text{Verify}_{\text{rekey}}(\pi_{\text{rekey}}; \text{pk}_U, \text{pk}'_U, (B_{U,[i]}.D, D'_i)_{i=0}^3) \triangleright$ each new handle is the old one transported to pk'_U under one shared witness
 - 11: **assert** $\mathcal{E}'$ verifies against $\text{pk}'_U, \mathcal{J}$ as in Algorithm 1
 - 12: **for** $i = 0$ **to** 3 **do**
 - 13: $B_{U,[i]} \leftarrow (B_{U,[i]}.C, D'_i) \triangleright$ reuse the on-chain commitment unchanged, so by binding it still encodes the same v_i (whatever its magnitude under the current τ_{act}); only the handle is swapped, so no fresh range/EG proof is needed
 - 14: **end for**
 - 15: $\text{pk}_U \leftarrow \text{pk}'_U$; store the encapsulation ciphertexts of $\mathcal{E}'$
-

6 Universal-Composability Security

We give a single security statement in the universal-composability (UC) framework [Can01]: the protocol Π_{ConfTx} of Section 5 UC-realizes an ideal functionality $\mathcal{F}_{\text{ConfTx}}$ (Figure 2) that maintains confidential state in plaintext and exposes a small, explicit leakage profile to the simulator $\mathcal{S}$. Two properties follow directly from the functionality.

- *Integrity.* Every accepted transition is value-preserving by construction, so no environment can drive $\mathcal{F}_{\text{ConfTx}}$ into inflation, over-spend, or a negative balance.
- *Privacy.* $\mathcal{S}$ sees only the leakage column of Figure 2: transfer amounts between honest parties are hidden, and only the public boundary crossings (wrap, unwrap) together with the visible transaction graph are revealed.

Both Π_{ConfTx} and $\mathcal{F}_{\text{ConfTx}}$ are parameterized by a UC session identifier sid (Section 6.1); the realization claim is stated per sid (Theorem 3) and lifted to concurrent execution as Corollary 4.

6.1 Model

UC at a glance. Parties are PPT machines scheduled by a PPT environment $\mathcal{Z}$. Π_{ConfTx} runs in the $\mathcal{F}_{\text{Ledger}}$ -hybrid model alongside a real adversary $\mathcal{A}$; in the ideal world, $\mathcal{F}_{\text{ConfTx}}$ runs alongside $\mathcal{S}$. Π UC-realizes $\mathcal{F}$ if for every PPT $\mathcal{A}$ there exists a PPT $\mathcal{S}$ such that $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} \stackrel{c}{\approx} \text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$ for every PPT $\mathcal{Z}$.

Hybrid functionalities. The realization uses two hybrid functionalities: the ledger $\mathcal{F}_{\text{Ledger}}$, and the Groth–Ostrovsky–Sahai NIZK functionality $\mathcal{F}_{\text{NIZK}}$ [GOS06], one instance per proof relation, through which all transaction-level NIZKs are produced and verified.

Ledger. $\mathcal{F}_{\text{Ledger}}$ models Sui’s totally-ordered ledger: it accepts well-formed transactions, totally orders them, applies their state effects atomically, and exposes a single immutable public log. Replay protection lives in the ledger, so cryptographic transcripts in Π need no per-account version counter. Following [CDPW07], $\mathcal{F}_{\text{Ledger}}$ is a *global* (shared) hybrid: a single ledger instance is available to every session of Π_{ConfTx} and to $\mathcal{Z}$ directly, modeling that all ConfTx deployments on Sui share the same chain. Within $\mathcal{F}_{\text{Ledger}}$, each session’s on-chain state lives under its own sid , so different sessions’ state never interferes.

Proof systems. All transaction-level NIZKs run in the $\mathcal{F}_{\text{NIZK}}$ -hybrid model (Figure 1), the NIZK functionality of Groth, Ostrovsky, and Sahai [GOS06], with one instance $\mathcal{F}_{\text{NIZK}}^R$ per relation. We use two properties of $\mathcal{F}_{\text{NIZK}}$ in place of standalone zero-knowledge, soundness, and extractability. A proof is produced from the statement alone, so $\mathcal{S}$ simulates every honest-party proof without a witness, including the false DDH balance statements of the ideal world; this simulation is perfect. Additionally, $\mathcal{F}_{\text{NIZK}}$ accepts a proof only once a witness for its relation has been supplied, so $\mathcal{S}$ obtains a valid witness for every corrupt-prover proof the ledger accepts, with no rewinding.

Well-formedness is guaranteed by combining proofs: the protocol submits several $\mathcal{F}_{\text{NIZK}}$ instances that share the same Pedersen limb commitments. Here $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$ is the instance for the witness-folded relation $\mathcal{R}_{\text{EG}}^{\otimes}$ (Section 2); one proof covers all ciphertexts under one key, and its witness carries one opening (m_j, r_j) per ciphertext. A four-limb amount is pinned by $\mathcal{F}_{\text{NIZK}}^{\text{range}}$ (16-bit) and $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$; an eight-limb auditor-keyed encapsulation by $\mathcal{F}_{\text{NIZK}}^{\text{range}}$ (32-bit) and $\mathcal{F}_{\text{NIZK}}^{\text{KC}}$. This is the pattern used throughout: each instance extracts its own opening of a shared commitment, and Pedersen binding under discrete log forces the openings to agree, gluing the instances into one witness. That agreement is the only fact the realization needs beyond $\mathcal{F}_{\text{NIZK}}$; with the DDH ciphertext switches it is the whole

Functionality $\mathcal{F}_{\text{NIZK}}$

Parameterized with a relation R and running with parties $P_1, \dots, P_n$ and adversary $\mathcal{S}$.

- **Proof.** On input $(\text{prove}, \text{sid}, \text{ssid}, x, w)$ from a party P , ignore if $(x, w) \notin R$. Send (prove, x) to $\mathcal{S}$ and wait for the answer π ; store (x, π) and send $(\text{proof}, \text{sid}, \text{ssid}, \pi)$ to P .
- **Verification.** On input $(\text{verify}, \text{sid}, \text{ssid}, x, \pi)$ from a party V , check whether (x, π) is stored. If not, send (verify, x, π) to $\mathcal{S}$ and wait for the answer $(\text{witness}, w)$; upon receiving it, check whether $(x, w) \in R$ and, in that case, store (x, π) . If (x, π) has been stored return $(\text{verification}, \text{sid}, \text{ssid}, 1)$ to V , else return $(\text{verification}, \text{sid}, \text{ssid}, 0)$.

Figure 1: The NIZK argument functionality $\mathcal{F}_{\text{NIZK}}$ ([GOS06], Fig. 3), parameterized by the relation R . The protocol invokes one instance per proof relation $(\mathcal{R}_{\text{DDH}}, \mathcal{R}_{\text{EG}}^{\otimes}, \mathcal{R}_{\text{range}}^w, \mathcal{R}_{\text{KC}}, \mathcal{R}_{\text{rekey}})$.

cryptographic content of the proof, surfacing as the binding aborts E_{WEA} , E_{EG} , and E_{WKE} . Each is a simulator abort raised when the two extracted openings of a shared commitment disagree, breaking Pedersen binding and hence bounded by the discrete-log advantage; we define them precisely at the hops where they arise in the indistinguishability proof below.

Heuristic instantiation. A UC NIZK functionality cannot be realized in the plain model, so $\mathcal{F}_{\text{NIZK}}$ requires a trusted setup: it admits a standard-model realization in the common-reference-string hybrid [GOS06], and is also realizable in variants of the random oracle hybrid [CJS14, LR22]. Our deployment follows the latter route, heuristically instantiating each $\mathcal{F}_{\text{NIZK}}^R$ by the Fiat–Shamir transform [FS87] of the underlying Σ -protocol $(\mathcal{R}_{\text{DDH}}, \mathcal{R}_{\text{EG}}^{\otimes}, \mathcal{R}_{\text{KC}}, \mathcal{R}_{\text{rekey}})$ or of Bulletproofs $(\mathcal{R}_{\text{range}})$, with a domain-separated hash as the random oracle. Two results partially support this instantiation. Faust et al. [FKMV12] prove the Fiat–Shamir transform of a Σ -protocol with quasi-unique responses simulation-sound and weakly simulation-extractable, and Ganesh et al. [GOP⁺22] prove straight-line simulation-extractability of Fiat–Shamir Bulletproofs in the algebraic group model [FKL18]. These results are model-dependent: the random oracle is used both to derive Fiat–Shamir challenges for soundness and, via programmability, in zero-knowledge simulation, while AGM straight-line extraction only covers algebraic adversaries. They therefore provide evidence for the heuristic instantiation, not a standard-model UC realization of the deployed proofs.

Session identifiers. Following the UC convention, both $\Pi_{\text{ConfTx}}^{\text{sid}}$ and $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ are parameterized by a session identifier sid chosen by $\mathcal{Z}$ at session start. Every input from $\mathcal{Z}$, every leakage to $\mathcal{S}$, and every message exchanged with $\mathcal{F}_{\text{Ledger}}$ is tagged with sid , and an instance silently drops mismatched tags. Within a single sid , the functionality maintains independent per-token and per-account state; distinct sids correspond to disjoint protocol instances.

The Fiat–Shamir transcript label sid used inside the NIZKs of Appendix A is a distinct object from the UC session identifier; the former is derived deterministically from the latter together with the holder identity and a one-byte protocol tag (Appendix A.1). We overload notation since the surrounding context fixes the intended meaning at every occurrence.

No sub-session identifier. Unlike several functionalities, such as UC-(A)PAKE [GMR06, JR18], which add a sub-session identifier ssid on top of sid to defeat replay across sub-sessions sharing a long-lived random-oracle output (the password hash), $\Pi_{\text{ConfTx}}^{\text{sid}}$ needs none: each on-chain NIZK statement is freshly randomized per invocation by the holder’s blindings, giving a statement-fresh Fiat–Shamir transcript, and full-transaction replay is ruled out separately by $\mathcal{F}_{\text{Ledger}}$. The detailed argument appears in Appendix A.1.

Corruption. Corruption is static: at the start of execution, $\mathcal{Z}$ designates a set $\mathcal{H}^*$ of corrupt holders. The issuer is folded into $\mathcal{Z}$, since it holds no per-account decryption key. *Auditors are assumed honest.* A deployment with corrupt auditors can be modeled by adding every account whose encapsulation lists a corrupt auditor key to $\mathcal{H}^*$ on the leakage side; supporting that natively would require a forced public-key refresh on every auditor-list update, which we rule out on usability grounds.

6.2 The ideal functionality

$\mathcal{F}_{\text{ConfTx}}$ is an ideal functionality that maintains plaintext per-account state for its token. We describe its state, the conventions we follow, and the built-in invariants in the paragraphs below; the per-input rules (validity predicate, plaintext-state update on accept, and message delivered to the simulator $\mathcal{S}$ as that input’s leakage) are enumerated in Figure 2.

State. The state below belongs to a single session $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$; the *sid* index is implicit on every variable and omitted for legibility. State is disjoint across sessions, since both $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ and its on-chain prefix in $\mathcal{F}_{\text{Ledger}}$ are *sid*-indexed (Section 6.1). The session identifier *sid* fixes a single token type T , a static parameter of the functionality known up front (like the party set) rather than created at runtime, so $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ governs one token. For this T the functionality maintains a non-negative pool $\text{Pool}_T \in \mathbb{Z}_{\geq 0}$, the current auditor list $\mathcal{J}_T$ with version number ν_T , and the history of past auditor lists $\mathcal{J}_T^{[v]}$ for every $v \leq \nu_T$ (consulted by the **Audit** rule of Figure 2). For each registered account U (honest or corrupt) under token T , it maintains active and pending balances $\text{act}_U^T, \text{pnd}_U^T \in \mathbb{Z}_{\geq 0}$ and an encapsulation-version index $\nu_U^T \in \mathbb{N}$ recording the value of ν_T at U ’s most recent **Register**/**Rotate**. Per-account key material lies intentionally outside the functionality: a holder’s public key, its viewing-key encapsulation, and any associated proof live on the simulated ledger maintained by $\mathcal{S}$. Whether U ’s on-chain encapsulation is non-empty is determined by the auditor list in force at the account’s encapsulation version, $\mathcal{J}_T^{[\nu_U^T]}$, so no separate flag is recorded. Amount parameters a and v are drawn from the u64 input alphabet $[0, 2^{64})$.

Conventions. Each row of Figure 2 specifies, for one input, the validity predicate, the plaintext-state update on accept, and the *leakage* that $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ delivers to $\mathcal{S}$ (in Canetti’s syntax [Can01], the message $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ sends to $\mathcal{S}$ on each accepted input; for the gated rows introduced next, the same message is delivered as the gate query, before acceptance). Leakage tuples in the right column are implicitly *sid*-prefixed; we omit the prefix for brevity. When the table or surrounding prose says $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ *emits* the marker **Failed**, the marker is included in the leakage and $\mathcal{S}$ realizes it as a public-log entry under the *sid*-prefixed object of $\mathcal{F}_{\text{Ledger}}$, observable to every party. The **AuditUnavailable** marker of the **Audit** row is different: auditing is an off-chain read (Section 3.3), so the marker is returned to the querying auditor alone, enters no leakage, and leaves no public-log entry.

Availability gate. The rows of Figure 2 marked *gated* (**Wrap**, **Transfer**, and **Rotate**) are subject to an additional acceptance step. Once the row’s validity predicate (if any) holds, $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ delivers the row’s leakage to $\mathcal{S}$ and waits for a reply in $\{\text{ok}, \text{reject}\}$; on **ok** it applies the update, while on **reject** it leaves the state unchanged and emits **Failed**. A block of same-sender transfers invoked atomically is gated as a unit: the leakage tuples of the block are delivered together, and the block is discarded with a single **Failed** marker if any pairwise validity predicate fails or $\mathcal{S}$ replies **reject**. The gate delegates to the adversary the availability of the gated inputs, and only their availability: a **reject** does not mutate state, so the conservation invariant below is unaffected, and the gate query

consists of the leakage the row grants in any case, so $\mathcal{S}$ receives no additional information. The gate serves to keep the ledger’s public bookkeeping outside the functionality. The real ledger refuses a transfer credit or a wrap that violates the counter invariant of Section 3.1, and refuses a key rotation while pending encrypted deposits exist. Each of these predicates is a deterministic function of the public transaction schedule and is independent of the hidden amounts. The simulator answers a gate query by evaluating the corresponding check on its simulated ledger (*gate accounting*, Section 6.3), so an input is rejected in the ideal world if and only if it is rejected in the real one. For inputs that $\mathcal{S}$ submits on behalf of corrupt parties the gate is vacuous, as $\mathcal{S}$ forwards only transactions that its simulated ledger has accepted. Finally, the leakage-only row **Normalize** (the update-balance operation of Algorithm 7) carries no plaintext update; it is included in the interface so that honest update-balance invocations appear on the simulated public log and enter the gate accounting.

Conservation invariant. After every accepted input, $\sum_U (\text{act}_U^T + \text{pnd}_U^T) = \text{Pool}_T$, which forces $\text{act}_U^T, \text{pnd}_U^T \leq \text{Pool}_T < 2^{64}$ for every registered U . The pool bound holds because **Wrap**, the only input that increases Pool_T , carries the validity predicate $\text{Pool}_T + v < 2^{64}$; this mirrors the real pool, a u64 **Balance** whose arithmetic aborts on overflow, and both sides of the predicate are determined by the public wrap and unwrap amounts, so it is evaluated identically in the two worlds. The base case is **InitToken**, after which both sides are zero. For the inductive step, each transition either (i) moves v between Pool_T and a single account, (ii) redistributes a between two registered accounts without touching Pool_T (**Transfer** debits act_S^T and credits pnd_R^T , including the corrupt-corrupt case invoked by $\mathcal{S}$ on behalf of S^*), (iii) rebalances within a single account (**Merge** folds pnd_U^T into act_U^T), or (iv) does not touch any value variable (**Register**, **Rotate**, **Normalize**, **UpdateAuditors**, and **Audit**). A gate **reject** leaves the state unchanged, so gated inputs preserve the invariant on both branches. Non-negativity follows from the sender-funds-available preconditions $a \leq \text{act}_S^T$ and $v \leq \text{act}_U^T$ on **Transfer** and **Unwrap**, since the value variables are plaintext integers maintained by $\mathcal{F}_{\text{ConfTx}}$. The amount range $[0, 2^{64})$ is an input-alphabet property; encryption and proof-system soundness play no role at this level.

Leakage classes. The leakage column partitions inputs into three classes. *Public-by-design* (**InitToken**, **Wrap**, **Unwrap**, **UpdateAuditors**): every parameter, including amounts, is delivered to $\mathcal{S}$ because these operations cross the public/private boundary or alter publicly observable token configuration. *Identity-only* (**Transfer** between honest parties, honest **Register**/**Rotate**/**Merge**/**Normalize**): only the participating identifiers and the existence of the operation are revealed; amounts remain hidden. *Fully private* (**Audit**): nothing is delivered to $\mathcal{S}$, but the auditor’s input and output sit between $\mathcal{Z}$ and $\mathcal{F}$, so the returned value is *observable to $\mathcal{Z}$* : $\mathcal{Z}$ supplies the query (T, U) on behalf of an honest A_j and reads back either $(\text{act}_U^T, \text{pnd}_U^T)$ or the **AuditUnavailable** marker. This is what makes the **Audit** row substantive: matching the two worlds on A_j ’s output certifies auditor *correctness* (a registered auditor recovers the true balance of an honestly registered account) and auditor *soundness* (a corrupt registrant cannot make the recovered balance disagree with $\mathcal{F}$, i.e. cannot evade audit). The simulator is never notified of audit queries; agreement between the two worlds is established in Section 6.3 and relies on auditors being honest, so sk_{A_j} never enters $\mathcal{Z}$ ’s view. There is no tension with the identity-only **Transfer** row: amount hiding is a guarantee against the network adversary $\mathcal{S}$, not against a key-holding auditor, and $\mathcal{Z}$ acting through an honest A_j may recover both endpoints’ balances (hence, by differencing across a transfer, its amount). This disclosure is identical in the two worlds ($\mathcal{F}$ returns the same balances that the real auditor decrypts), so it grants $\mathcal{Z}$ no distinguishing power.

Corruption modifies the identity-only row: **Transfer** when S or R is corrupt forwards the amount,

Functionality $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$		
Input	Validity (<i>italic</i>) and update on accept	Leakage to $\mathcal{S}$
InitToken(T) from issuer	T unset: $\text{Pool}_T \leftarrow 0, \mathcal{J}_T \leftarrow \emptyset, \nu_T \leftarrow 0, \mathcal{J}_T^{[0]} \leftarrow \emptyset$	(T)
UpdateAuditors($T, \mathcal{J}_T^{\text{new}}$) from issuer	$\mathcal{J}_T \leftarrow \mathcal{J}_T^{\text{new}}, \nu_T += 1$, record $\mathcal{J}_T^{[\nu_T]} \leftarrow \mathcal{J}_T^{\text{new}}$. May be invoked atomically with InitToken, so no point in time has $\mathcal{J}_T = \emptyset$.	($T, \mathcal{J}_T^{\text{new}}$)
Register(T) from U	U unregistered: $\text{act}_U^T, \text{pnd}_U^T \leftarrow 0; \nu_U^T \leftarrow \nu_T$.	(U, T, ν_T)
Rotate/RefreshEncap(T) from U	<i>Gated</i> : new incarnation under $\mathcal{J}_T$, preserving act_U^T and $\text{pnd}_U^T; \nu_U^T \leftarrow \nu_T$. On reject: emit Failed.	(U, T, ν_T) Failed if rejected
Wrap(T, U, v) from payer P	$\text{Pool}_T + v < 2^{64}$, <i>gated</i> . On accept: $\text{Pool}_T += v; \text{pnd}_U^T += v$. Else reject; emit Failed.	(P, U, v) Failed if rejected
Transfer(T, R, a) from S	$a \leq \text{act}_S^T$, <i>gated</i> . On accept: $\text{act}_S^T -= a; \text{pnd}_R^T += a$. Else reject; emit Failed. Several same-sender transfers can be invoked atomically (gated as one block).	(S, R); a if $\{S, R\} \cap \mathcal{H}^* \neq \emptyset$ Failed if rejected
Unwrap(T, v) from U	$v \leq \text{act}_U^T$. On accept: $\text{Pool}_T -= v; \text{act}_U^T -= v$. Else reject; emit Failed.	(U, v) Failed if rejected
Merge(T) from U	$\text{act}_U^T += \text{pnd}_U^T; \text{pnd}_U^T \leftarrow 0$.	(U)
Normalize(T) from U	No plaintext update.	(U)
Audit(T, U) from A_j	$A_j \in \mathcal{J}_T^{[\nu_U^T]}$: return $(\text{act}_U^T, \text{pnd}_U^T)$ to A_j . Else reject; return AuditUnavailable to A_j .	none

Figure 2: Per-input interface of $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$: validity predicate, plaintext-state update on accept, and leakage delivered to the simulator. Inputs and leakage tuples are implicitly *sid*-tagged; state, generic interface loop, and built-in invariants are described in the prose preceding this figure.

and corrupt-only Register/Rotate/Merge/Normalize inputs invoked by $\mathcal{S}$ on behalf of $\mathcal{H}^*$ forward the full input tuple. This is unavoidable: a corrupt sender already knows the amount it transfers, a corrupt receiver decrypts its own incoming ciphertext, and corrupt-only operations are initiated by $\mathcal{A}$ itself.

Out-of-scope: privileged issuer paths. The privileged issuer paths of Section 4 (global pause, per-account deny list, direct balance overwrite) are not modeled in $\mathcal{F}_{\text{ConfTx}}$. The UC realization claim (Theorem 3) assumes the issuer does not invoke them or maintains the plaintext-balance invariants tracked by $\mathcal{F}_{\text{ConfTx}}$ in case it does.

6.3 Realization

Theorem 3 (UC realization). *Assume the DDH assumption holds on $\mathbb{G}$ (which implies hardness of discrete log, hence binding of the Pedersen commitment), and model the transfer key-derivation hash KDF as a random oracle. Then the protocol $\Pi_{\text{ConfTx}}^{\text{sid}}$ UC-realizes $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ in the $(\mathcal{F}_{\text{Ledger}}, \mathcal{F}_{\text{NIZK}})$ -hybrid model under static corruption.*

We fix an arbitrary sid , exhibit a per-session simulator $\mathcal{S} = \mathcal{S}^{\text{sid}}$, and show that no PPT $\mathcal{Z}$ distinguishes the real-world execution of $\Pi_{\text{ConfTx}}^{\text{sid}}$ from the ideal-world execution it induces with more than negligible advantage. References to “the ledger” below mean the sid -prefixed slice of $\mathcal{F}_{\text{Ledger}}$.

Simulator $\mathcal{S}$. $\mathcal{S}$ runs $\mathcal{A}$ internally, maintains a simulated $\mathcal{F}_{\text{Ledger}}$, and reacts to each leakage event of $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ by extending the simulated ledger with a corresponding sid -tagged transaction.

Honest holders. For each new honest holder U , $\mathcal{S}$ samples a fresh keypair $(\text{sk}_U^{\mathcal{S}}, \text{pk}_U^{\mathcal{S}})$, publishes $\text{pk}_U^{\mathcal{S}}$, and maintains all of U ’s transfer-side ciphertexts (active and pending balances, per-receiver amounts, and the aggregate sender-side handle) and the auditor-keyed encapsulation as encryptions of zero. For a transfer it samples the ephemeral scalar e itself, publishes $P = eG$, and derives the per-limb blindings from $Z_S = e \text{pk}_U^{\mathcal{S}}$ exactly as an honest sender would, so the simulated blindings follow the real distribution without $\text{sk}_U^{\mathcal{S}}$; the aggregate handle is $\hat{A}_S.D = r_S \text{pk}_U^{\mathcal{S}}$ for the collapsed blinding r_S it thereby knows. Every accompanying proof is produced through $\mathcal{F}_{\text{NIZK}}$ ’s proof interface on the statement alone. The sampled $\text{sk}_U^{\mathcal{S}}$ only fixes the published key and enters no ciphertext, so $\mathcal{S}$ never uses an honest secret key.

Corrupt holders. $\mathcal{S}$ relays $\mathcal{A}$ ’s ledger submissions verbatim and forwards each accepted transaction to $\mathcal{F}_{\text{ConfTx}}$. For an honest-sender **Transfer** to a corrupt R^* , $\mathcal{F}_{\text{ConfTx}}$ leaks the amount and $\mathcal{S}$ encrypts it under pk_{R^*} . For a corrupt-sender **Transfer**, $\mathcal{S}$ reads the per-receiver witness $(a_{k,i}, \rho_{k,i})_{i=0}^3$ (the openings of the well-formed encrypted amount A_{R_k} , each limb $A_{R_k,[i]} = \text{Enc}(\text{pk}_{R_k}, a_{k,i}; \rho_{k,i})$ with $a_{k,i} \in [0, 2^{16})$) off the $\mathcal{F}_{\text{NIZK}}^{\text{EG}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$ interface and sets $a_k = \sum_i 2^{16i} a_{k,i} \in [0, 2^{64})$, using no receiver secret key. A corrupt **Rotate** is accepted only with the re-key checks $(\text{pk}'_{U^*} \neq \mathcal{O}$, the reused on-chain commitments $B_{U^*,[i]}.C$ with only the re-keyed handles $(D'_i)_{i=0}^3$ posted, the batched $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$ proof binding $\text{pk}'_{U^*} = w \text{pk}_{U^*}$ and $D'_i = w B_{U^*,[i]}.D$, and a fresh encapsulation $\mathcal{E}'$ of sk'_{U^*}), which preserve the audited balance: commitment reuse fixes the plaintext by binding while the proof only rebases the handles to pk'_{U^*} .

Gate accounting. $\mathcal{S}$ answers the availability gates of Figure 2 by evaluating, on its simulated ledger, the same public-bookkeeping checks the real ledger applies. Per account it tracks the bound counters $\tau_{\text{act}}, \tau_{\text{pnd}}$ and the public accumulator b_{pub} , updated on each accepted input exactly as the counter invariant of Section 3.1 prescribes; every update is a deterministic function of the leakage stream. $\mathcal{S}$ replies ok exactly when the ledger’s precondition holds, evaluated in submission order: for each **Transfer** credit to R , the slot condition $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$ (Algorithm 5); for a **Wrap** to U , $b_{\text{pub}}^U > 0$ or the same slot condition; for a **Rotate** by U , $\tau_{\text{pnd}}^U = 0$ (Algorithm 8). These quantities are determined by the public transaction schedule, identical in the two worlds, so the gated inputs accepted ideally are exactly those accepted on-chain.

Audit. An **Audit** is answered by $\mathcal{F}_{\text{ConfTx}}$ directly, so $\mathcal{S}$ takes no action; what must hold is that the auditor output read by $\mathcal{Z}$ agrees across worlds. For an honestly registered U the ideal output is $\mathcal{F}$ ’s plaintext $(\text{act}_U^T, \text{pnd}_U^T)$, which the real auditor recovers by decrypting U ’s balances while they still encrypt their true values; the later switch of these ciphertexts to zero (G_4 – G_5) is invisible because the audit answer is the tracked plaintext, and it is sound only because auditors are honest, so sk_{A_j} never enters $\mathcal{Z}$ ’s view. For a corrupt U^* , hop G_3 shows that the accepted encapsulation pins

sk_{U^*} , so the auditor's bounded brute-force decryption returns the same balance as $\mathcal{F}$. Multi-receiver batched transfers reduce to K pairwise applications of these cases.

Proof of indistinguishability. We pass from the real execution G_0 to the ideal execution G_6 in six hops: three witness extractions, two DDH switches, and one information-theoretic step. G_0 is Π_{ConfTx} in the $(\mathcal{F}_{\text{Ledger}}, \mathcal{F}_{\text{NIZK}})$ -hybrid model; honest proofs already come from $\mathcal{F}_{\text{NIZK}}$ on the statement alone, so no separate zero-knowledge hop is needed. The extraction hops read the $\mathcal{F}_{\text{NIZK}}$ witnesses of accepted corrupt proofs and each adds one binding abort: G_1 the $\mathcal{F}_{\text{NIZK}}^{\text{EG}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$ witnesses of corrupt amounts (E_{WEA}), G_2 the opening of the aggregate ciphertext $\hat{A}_S$ carried in the same folded $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$ witness (E_{EG}), and G_3 the $\mathcal{F}_{\text{NIZK}}^{\text{KC}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$ opening of each corrupt encapsulation (E_{WKE}). Outside the aborts these hops leave the view unchanged and equip the challenger with every corrupt plaintext, so the two DDH reductions that follow run without the challenged secret keys and never decrypt: corrupt parties' contributions come from the extracted witnesses. G_4 switches the honest auditor-keyed encapsulations and G_5 the honest transfer-side ciphertexts to encryptions of zero. This order is forced: the G_5 reduction does not know the challenged holder's sk_U , while the protocol publishes an encapsulation of sk_U ; after G_4 that encapsulation encrypts zero, so the reduction need not produce it. The final hop $G_5 \rightarrow G_6$ is information-theoretic: with the extracted openings and the balance relation $\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$ certifies for every accepted spend, each accepted transition is value-preserving and is forwarded to $\mathcal{F}_{\text{ConfTx}}$.

$G_0 \rightarrow G_1$ (*encrypted amounts*). Read the $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$ and $\mathcal{F}_{\text{NIZK}}^{\text{range}}$ witnesses of every accepted corrupt amount: the new balance B'_{S^*} in transfers, unwraps, balance update, and each receiver amount A_{R_k} . Add the abort event E_{WEA} , raised when the two openings of some shared limb commitment C_i disagree; this breaks Pedersen binding, so $\Pr[E_{\text{WEA}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$. On $\neg E_{\text{WEA}}$ each freshly range-proven limb lies in $[0, 2^{16}]$ (otherwise, an accumulated balance limb is only bounded by 2^{32}) over $\mathbb{Z}$ and satisfies $D_i = r_i \text{pk}$, so the balance relation lifts to $\mathbb{Z}$ without wrap-around and the recovered amount $a_k = \sum_i 2^{16i} m_{k,i}$ lies in $[0, 2^{64}]$.

$G_1 \rightarrow G_2$ (*aggregate sender consistency*). For every accepted corrupt transfer, read the opening $(r_{\hat{A}}, m_{\hat{A}})$ of the aggregate ciphertext $\hat{A}_S$ from the sender-side folded $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$ witness (the fifth slot of γ_S , alongside the four new-balance limb openings read at G_1), with $\hat{A}_S = (r_{\hat{A}}G + m_{\hat{A}}H, r_{\hat{A}}\text{pk}_{S^*})$. Add the abort event E_{EG} , raised when this opening of $\hat{A}_S.C$ disagrees with the one forced by the ledger's reconstruction $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$ and the G_1 openings of the A_{R_k} ; again this breaks binding, so $\Pr[E_{\text{EG}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$. On $\neg(E_{\text{EG}} \cup E_{\text{WEA}})$ the openings agree, fixing $m_{\hat{A}} = a = \sum_k a_k$ and pinning the aggregate handle to $\hat{A}_S.D = (\sum_{k,i} 2^{16i} \rho_{k,i}) \text{pk}_S$.

$G_2 \rightarrow G_3$ (*encapsulation well-formedness*). Add the abort event E_{WKE} , raised when the $\mathcal{F}_{\text{NIZK}}^{\text{range}}$ and $\mathcal{F}_{\text{NIZK}}^{\text{KC}}$ openings of some limb commitment C_i of an accepted corrupt encapsulation disagree. Binding gives $\Pr[E_{\text{WKE}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$. On $\neg E_{\text{WKE}}$ the candidate opening that the multi-recipient handles uniquely determine ($r_i = \log_{\text{pk}_{A_j}} D_{ij}$ for any honest A_j , then $u_i = \log_H(C_i - r_i G)$) lies in L_{WKE} : $u_i \in [0, 2^{32})$ over $\mathbb{Z}$ and $\text{sk}_{U^*} = \sum_i 2^{32i} u_i$ with $\text{pk}_{U^*} = \text{sk}_{U^*} G$. The real auditor's bounded brute-force decryption recovers the same u_i (uniquely, as $q \gg 2^{32}$) and the same balance, so the audit row of $\mathcal{F}_{\text{ConfTx}}$ agrees with the real world on every corrupt U^* .

$G_3 \rightarrow G_4$ (*encapsulation switch*). Switch every honest holder's auditor-keyed encapsulation $C_i = (r_i G + u_i H, (r_i \text{pk}_{A_j})_{j=1}^n)$, $i = 0, \dots, 7$, from an encryption of the 32-bit limbs of sk_U under $\mathcal{J}$ to an encryption of zero, reusing the shared per-limb blinding r_i of the multi-recipient scheme (Appendix A.4). Over all honest Register/Rotate limbs the hop is bounded by $\text{poly}(\lambda) \cdot \text{Adv}_{\mathbb{G}}^{\text{DDH}}$. The hop is IND-CPA security of the multi-recipient encapsulation: it differs from an encryption of zero

only in the $u_i H$ term of the shared commitment, the handles $r_i \text{pk}_{A_j}$ being identical. For an honest slot j^* the reduction embeds a DDH instance $(G, \text{pk}_{A_{j^*}} = \alpha G, X, Y)$ and posts $C_i = X + u_i H$, $D_{i,j^*} = Y$, and $D_{ij} = \text{sk}_{A_j} X$ for $j \neq j^*$, sampling the other honest keys itself. A real tuple $(Y = \alpha \log_G X)$ gives the encapsulation of u_i with blinding $\log_G X$ (G_3); a uniform Y makes D_{i,j^*} independent of C_i . Iterating over the n honest slots decouples every handle, after which the perfectly hiding C_i is the only term carrying the message, so the encapsulation is independent of u_i and indistinguishable from the encryption of zero the simulator posts in G_4 . The accompanying $\mathcal{F}_{\text{NIZK}}^{\text{KC}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$ proofs are on the statement alone, hence unaffected. Audit queries are also unaffected: the reduction samples every holder key itself and answers from the tracked plaintexts, honest via sk_U and corrupt via the sk_{U^*} pinned at G_3 , so no auditor secret key is needed. The eight limbs collapse to one challenge per slot by random self-reducibility, and a hybrid over the polynomially many honest Register/Rotate encapsulations gives the $\text{poly}(\lambda) \cdot \text{Adv}_{\mathbb{G}}^{\text{DDH}}$ bound.

$G_4 \rightarrow G_5$ (*transfer-side switch*). For every honest sender we first replace the KDF-derived transfer blindings by fresh uniform ones, which is invisible to $\mathcal{A}$: the ephemeral tuple (G, P, pk_S, Z_S) is a DDH instance, so $Z_S = \text{sk}_S P$ is indistinguishable from a uniform group element, after which $\mathcal{A}$ queries the random oracle KDF at Z_S only with probability $\leq Q/q$, so the outputs $\rho_{k,i} = \text{KDF}(Z_S, \cdot)$ are uniform and independent in its view. With fresh blindings the honest transfer matches the pre-derivation distribution, and we replace every honest-key transfer-side ciphertext by an encryption of zero: the new balance B'_S and, for each honest receiver, the amount A_{R_k} together with its sender-side handle. Sharing a per-limb commitment, receiver handle and sender handle form one $n = 2$ multi-recipient ciphertext under $(\text{pk}_{R_k}, \text{pk}_S)$ (Appendix A.4); honest Rotate is identical. The reduction embeds the DDH challenge in the two honest keys, holding neither sk_R nor sk_S , posts each A_{R_k} from the receiver-key components, and assembles the single aggregate handle $\hat{A}_S.D$ as the 2^{16i} -weighted sum of the sender-key components, so $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$ holds by construction. The challenged holders' auditor-keyed encapsulations already encrypt zero after G_4 , so the reduction posts them without their secret keys. Corrupt-sender amounts are read from the G_1 witnesses, and honest-to-corrupt amounts stay real encryptions under pk_{R^*} . Accompanying proofs are $\mathcal{F}_{\text{NIZK}}$ mediated on the statement alone, unaffected by the switch. The hop is bounded by $\text{poly}(\lambda) \cdot \text{Adv}_{\mathbb{G}}^{\text{DDH}} + Q/q$, the second term from the random-oracle guess on Z_S .

$G_5 \rightarrow G_6$ (*ideal world*). The simulator drops its plaintext bookkeeping and defers to $\mathcal{F}_{\text{ConfTx}}$, forwarding each accepted corrupt transaction with the amounts read at G_1 . Availability gates are answered by the gate accounting of the simulator description; since the gate predicates are deterministic functions of the public transaction schedule, identical across the two games, the gated accepts and Failed markers of G_6 coincide with the ledger's accepts and rejections in G_5 and contribute no advantage. The only obligation is that every accepted transition preserves value, which the two ledger-checked proofs guarantee: $\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$ for spends and $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$ for key rotation.

- *Spends* ($\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$). Registration and key rotation assert $\text{pk}_S \neq \mathcal{O}$, so $\text{sk}_S \neq 0$. For $\Delta = \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$ (or its unwrap or balance-update analogue), extraction yields x with $\text{pk}_S = xG$, forcing $x = \text{sk}_S$, and hence $\Delta.D = \text{sk}_S \Delta.C$. As $\text{sk}_S \neq 0$ the inverse sk_S^{-1} exists, so $\text{Dec}(\text{sk}_S, \Delta) = \log_H(\Delta.C - \text{sk}_S^{-1} \Delta.D) = \log_H(\Delta.C - \Delta.C) = 0$ and Δ encrypts zero. A degenerate $\text{pk} = \mathcal{O}$ account has $\text{sk} = 0$, so $\text{sk } m_\Delta H = \mathcal{O}$ holds for every m_Δ : any $\Delta = (\alpha G + m_\Delta H, \mathcal{O})$ with $m_\Delta \neq 0$ then verifies while shifting the balance by m_Δ . The G_1 and G_2 openings pin $\Delta.D$ to the true blindings, the range proofs keep b'_S and a in $[0, 2^{64})$, and the pre-state b_S satisfies $b_S < 2^{64}$ by the conservation invariant ($\text{act}_U^T + \text{pnd}_U^T \leq \text{Pool}_T < 2^{64}$) maintained inductively across accepted transitions rather than by a range proof in this transaction, so this lifts to $b'_S + a - b_S = 0$ over $\mathbb{Z}$ without wrap-around.

- *Rotation* ($\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$). B'_U reuses the commitments of B_U , so binding fixes the plaintext, and $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$ only rebases each handle to pk'_U , leaving the balance unchanged.

On $\neg(E_{\text{WEA}} \cup E_{\text{EG}} \cup E_{\text{WKE}})$ each accepted transition then matches its row of Figure 2, and conversely, so G_5 and G_6 are identically distributed.

Advantage bound. Only the DDH switches and the three binding aborts contribute; the random-oracle derivation of the transfer blindings adds a negligible guessing term Q/q once Z_S is switched, and zero-knowledge and knowledge soundness are intrinsic to the hybrid. Summing,

$$|\Pr[\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} = 1] - \Pr[\text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}} = 1]| \leq \text{poly}(\lambda)(\text{Adv}_{\mathbb{G}}^{\text{DDH}} + \text{Adv}_{\mathbb{G}}^{\text{dlog}}) + \text{negl}(\lambda),$$

where $\text{poly}(\lambda)$ counts the DDH and DL applications and $\text{negl}(\lambda)$ absorbs the random-oracle guessing terms. Since $\text{Adv}_{\mathbb{G}}^{\text{dlog}} \leq \text{Adv}_{\mathbb{G}}^{\text{DDH}}$, the probability is negligible under DDH in the random-oracle model, proving Theorem 3.

Concurrent composability. The per-session realization of Theorem 3 lifts to arbitrary concurrent execution by Canetti’s universal-composition theorem.

Corollary 4 (Concurrent composability). *Under the assumptions of Theorem 3, the multi-session extension of Π_{ConfTx} , in which arbitrarily many copies $\Pi_{\text{ConfTx}}^{\text{sid}_1}, \Pi_{\text{ConfTx}}^{\text{sid}_2}, \dots$ (each fixing a token type via its sid) run concurrently against the shared $\mathcal{F}_{\text{Ledger}}$ and against arbitrary other protocols sharing the same ledger, UC-realizes the multi-session extension of $\mathcal{F}_{\text{ConfTx}}$.*

7 Related Work

The line of work most relevant to us runs from research prototypes to deployed smart-contract systems. *Zether* [BAZB20] was the first homomorphic-ElGamal confidential-payments scheme inside a smart contract (on Ethereum); *Anonymous Zether* [Dia21] adds sender/receiver anonymity. *PGC* [CMTA20] fixed the cryptographic blueprint used by later systems: Twisted ElGamal, Σ -protocol balance proofs, Bulletproofs range proofs [BBB⁺18], and a two-layer auditing capability discussed below. That blueprint was implemented in Solana [Sol24] and adopted by later systems [Ste25, Fou26, CMA26].

Cryptographic primitives. Zether encrypts balances with symmetric “ElGamal in the exponent.” PGC introduced *Twisted ElGamal* [CMTA20], in which the ciphertext’s first component is a Pedersen commitment and only the second component (the decryption handle) depends on the recipient’s key; deployed systems adopted that blueprint with Bulletproofs range proofs on the commitment.

Account model and balance encoding. Zether and PGC store one ElGamal ciphertext per account, which limits practical balances to what baby-step/giant-step can recover from a single discrete log. Solana and later implementations instead split balances into small limbs, each encrypted separately, so per-limb decryption uses a small precomputed table.

Race-condition handling. On a concurrent ledger, a small incoming transfer can arrive after a sender builds a proof but before it executes, and invalidate that proof. Zether avoids this with fixed *epochs*: credits accumulate in a pending balance and roll into the spendable balance only at epoch boundaries, so senders face a deadline tied to epoch length. PGC does not address this race because it keeps a single balance ciphertext and has no pending/active split. Solana keeps separate pending and active (or available) balances and requires an explicit **merge** before spending pending funds.

Auditing. Zether provides no built-in auditing. PGC offers two optional layers: zero-knowledge policy proofs on each transfer (*regulation compliance*), and encryption of transfer amounts to a supervisor (*supervision*). Solana follows the supervision style at transfer time.

Security definitions and proofs. Zether states *overdraft-safety* (and anonymity for Anonymous Zether) in a game-based model. PGC states game-based soundness for its proofs and encryption. Solana documents its ZK-Token protocol in a two-part specification [Sol21a, Sol21b]: Part 1 gives a developer-facing protocol overview, and Part 2 provides game-based security definitions of a confidential payment abstraction.

Technical contributions of this work. Relative to recent implementations, we *inherit* the PGC-style core: Twisted ElGamal with Bulletproofs and Σ -protocol proofs, pending/active account state, and a backing pool with public `wrap/unwrap`. What follows is new in our design or formalization relative to that prior work:

1. *Per-account auditing with no per-transfer auditor data.* PGC’s supervision layer attaches auditor ciphertexts to *each* transfer. We instead publish, once per account at registration or key rotation, a multi-recipient Twisted ElGamal encapsulation of the holder’s secret viewing key sk_U to the issuer’s auditor list $\mathcal{J}$, certified by the new key-consistency proof Π_{KC} (Section 3.3). Listed auditors thereafter decrypt the full account view from the ledger, so later transfers carry no auditor ciphertexts or consistency proofs whatever the size of $\mathcal{J}$; the sole per-transfer field serving audit is the single ephemeral group element P , independent of the number of receivers.
2. *Optimistic merge-then-spend.* Merge-then-spend (Section 3.2, Algorithm 7) can commit a pending fold while the payout is retried, a completion mode the compared deployments do not offer.
3. *Explicit homomorphic-growth accounting.* Recent implementations use chunked limb encodings but do not expose a per-aggregate counter that tracks how much homomorphic addition has widened limb ranges and refuses further pending deposits when slack is exhausted. We attach an integer bound counter τ (in units of 2^{16}) to each encrypted aggregate and package range and ElGamal-consistency checks in the *well-formed encrypted amount* carrier of Definition 1; Sui programmable transactions let a user merge and transfer in one transaction while respecting this bound.
4. *UC security with an explicit leakage profile.* Prior work in this line states game-based overdraft-safety (Zether) or per-proof soundness (PGC). We define an ideal functionality $\mathcal{F}_{\text{ConfTx}}$ (Section 6) with explicit leakage per operation (including conservation between encrypted balances and the backing pool) and prove UC-realization, yielding compositional guarantees via Canetti’s theorem [Can01].

8 Advantages of Twisted ElGamal

The encryption scheme of Section 2 is the *twisted* variant of ElGamal due to Chen et al. [CMTA20], not the classical exponential ElGamal [ElG85] of Zether [BAZB20]. For a key pair (sk, pk) with $\text{pk} = \text{sk}G$, message m , and randomness r ,

$$\begin{aligned}\text{Enc}^{\text{cls}}(\text{pk}, m; r) &= (mG + r\text{pk}, rG), \\ \text{Enc}^{\text{twi}}(\text{pk}, m; r) &= (rG + mH, r\text{pk}).\end{aligned}$$

Both are additively homomorphic and IND-CPA under DDH; the consequential difference is the first component. The classical $C^{\text{cls}} = mG + r\text{pk}$ commits under bases (G, pk) whose relation $\text{pk} = \text{sk}G$ is *known* to the holder, who can therefore re-open it to any value via $(m+\delta, r-\delta/\text{sk})$. The twisted $C^{\text{twi}} = rG + mH$ is a Pedersen commitment [Ped92] under bases (G, H) of unknown relation: perfectly hiding, computationally binding, and independent of pk . The three protocol features below all rest on these two properties.

Range proofs apply to the ciphertext directly. Well-formedness (Definition 1) requires each 16-bit limb to lie in $[0, 2^{16})$, certified by a Bulletproof [BBB⁺18] over a list of Pedersen commitments. Under twisted ElGamal each $E_i.C$ is already such a commitment under the basis (G, H) , where H is fixed by hash-to-curve so no party knows its discrete log to G ; the verifier runs on the limb commitments lifted unchanged from the ciphertext, and a transfer with $K+1$ amounts batches all $4(K+1)$ of them into one proof. Classical ElGamal cannot use (G, pk) as the CRS at all: soundness requires the prover to know no relation among the generators, yet the holder is the prover and knows $\text{pk} = \text{sk}G$ (the re-opening above is exactly this failure). Zether’s workaround keeps the CRS as (G, H) , attaches to each amount an auxiliary commitment $\tilde{C} = mH + sG$, proves the range on $\tilde{C}$, and adds a Σ -proof that $\tilde{C}$ and C^{cls} open to the same m , costing one extra group element and one equality proof per amount.

Cross-key amount equality needs no plaintext-equality proof. A private transfer must account for each amount a_k under two keys: the receiver’s credit is encrypted under pk_{R_k} as A_{R_k} , while the sender’s aggregate debit is charged under pk_S ; see Algorithm 4. Because C^{twi} is independent of pk , the sender’s debit reuses the receiver blindings, so the two sides share the same commitments and differ only in their decryption handles ($r\text{pk}_S$ versus $r\text{pk}_{R_k}$). The sender therefore posts no separate debit ciphertext: the ledger reconstructs the aggregate debit commitment $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$ from the on-chain receiver commitments and pairs it with a single aggregate handle $\hat{A}_S.D = r_S\text{pk}_S$, tied to pk_S by one Π_{EG} proof and needing no plaintext-equality proof. Classical ElGamal cannot share commitments: $C^{\text{cls}} = mG + r\text{pk}$ depends on pk , so the same amount under two keys differs in both components and a full Σ -protocol of plaintext equality is required.

Multi-recipient encapsulation shares one commitment. Auditor key encapsulation (Section 3.3, Algorithm 1) publishes, once per account, eight multi-recipient ciphertexts decryptable by any of the n auditors. The shared commitment $C = rG + mH$ again sits under the trustless basis (G, H) , so a single key-consistency proof Π_{KC} over the blindings $(r_i)_{i=0}^7$ binds all eight commitments to sk_U at once. Classical nonce reuse matches the bare element count of $1 + n$ per limb, but it binds the $8n$ key-dependent commitments $C_{ij} = u_iG + r_i\text{pk}_{A_j}$ under the prover-known bases (G, pk_{A_j}) , which, as in the range-proof case, admit no direct certificate and force per-recipient auxiliary commitments and equality proofs.

References

- [BAZB20] Benedikt Bünz, Shashank Agrawal, Mahdi Zamani, and Dan Boneh. Zether: Towards privacy in a smart contract world. In Joseph Bonneau and Nadia Heninger, editors, *FC 2020*, volume 12059 of *LNCS*, pages 423–443. Springer, Cham, February 2020.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018*

- IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
 - [CDPW07] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 61–85. Springer, Berlin, Heidelberg, February 2007.
 - [CJS14] Ran Canetti, Abhishek Jain, and Alessandra Scafuro. Practical UC security with a global random oracle. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *ACM CCS 2014*, pages 597–608. ACM Press, November 2014.
 - [CMA26] Murat Cenk, Aanchal Malhotra, and Joseph A. Akinyele. Confidential transfers for multi-purpose tokens on the XRP ledger. Cryptology ePrint Archive, Paper 2026/602, 2026.
 - [CMTA20] Yu Chen, Xuecheng Ma, Cong Tang, and Man Ho Au. PGC: Decentralized confidential payment system with auditability. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part I*, volume 12308 of *LNCS*, pages 591–610. Springer, Cham, September 2020.
 - [CP93] David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 89–105. Springer, Berlin, Heidelberg, August 1993.
 - [Dia21] Benjamin E. Diamond. Many-out-of-many proofs and applications to anonymous zether. In *2021 IEEE Symposium on Security and Privacy*, pages 1800–1817. IEEE Computer Society Press, May 2021.
 - [dVGH⁺23] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, IETF, 2023.
 - [ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
 - [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.
 - [FKMV12] Sebastian Faust, Markulf Kohlweiss, Giorgia Azzurra Marson, and Daniele Venturi. On the non-malleability of the Fiat-Shamir transform. In Steven D. Galbraith and Mridul Nandi, editors, *INDOCRYPT 2012*, volume 7668 of *LNCS*, pages 60–79. Springer, Berlin, Heidelberg, December 2012.
 - [Fou26] Aptos Foundation. AIP-143: Confidential APT. Aptos Improvement Proposal, 2026. Status: Accepted.
 - [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.

- [GMR06] Craig Gentry, Philip MacKenzie, and Zulfikar Ramzan. A method for making password-based key exchange resilient to server compromise. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 142–159. Springer, Berlin, Heidelberg, August 2006.
- [GOP⁺22] Chaya Ganesh, Claudio Orlandi, Mahak Pancholi, Akira Takahashi, and Daniel Tschudi. Fiat-Shamir bulletproofs are non-malleable (in the algebraic group model). In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 397–426. Springer, Cham, May / June 2022.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, Berlin, Heidelberg, May / June 2006.
- [JR18] Charanjit S. Jutla and Arnab Roy. Smooth NIZK arguments. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part I*, volume 11239 of *LNCS*, pages 235–262. Springer, Cham, November 2018.
- [LR22] Anna Lysyanskaya and Leah Namisa Rosenbloom. Universally composable Σ -protocols in the global random-oracle model. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 203–233. Springer, Cham, November 2022.
- [Ped92] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 129–140. Springer, Berlin, Heidelberg, August 1992.
- [Sol21a] Solana Labs. Part 1: SPL ZK-Tokens protocol overview. Solana ZK-Token protocol paper, December 2021.
- [Sol21b] Solana Labs. Part 2: SPL ZK-Tokens proof of security. Solana ZK-Token protocol paper, December 2021.
- [Sol24] Solana Foundation. Confidential Transfer extension on Token-2022. Solana developer documentation, 2024.
- [Ste25] Stellar Development Foundation. stellar-confidential-token: Prototype for confidential tokens on Stellar. GitHub repository, 2025.

A Zero-Knowledge Proof Protocols

The security analysis (Section 6.1) invokes these proof systems through the $\mathcal{F}_{\text{NIZK}}$ -hybrid model: each is modeled as an instance of the [GOS06] NIZK functionality for its relation, so the UC proof draws zero-knowledge and straight-line witness extraction directly from $\mathcal{F}_{\text{NIZK}}$ rather than from standalone game-based properties. The heuristic realization of each $\mathcal{F}_{\text{NIZK}}$ instance by the Fiat–Shamir transform, with references to the relevant papers, is discussed in Section 6.1. In this section we establish the underlying standalone constructions and their proof-of-knowledge guarantees, for the five relations of Section 2:

1. Π_{DDH} is the Chaum–Pedersen [CP93] NIZK proof of knowledge for $\mathcal{R}_{\text{DDH}}$ (Section A.2, Figure 3);

2. Π_{EG} is a NIZK proof of knowledge for $\mathcal{R}_{\text{EG}}$, obtained by applying the Fiat–Shamir transform to the Σ -protocol Σ_{EG} (Section A.3, Lemma 1, Figure 4);
3. Π_{range} is an aggregate range proof of knowledge for $\mathcal{R}_{\text{range}}^w$ ($w \in \{16, 32\}$), instantiated by Bulletproofs [BBB⁺18];
4. Π_{KC} is a NIZK proof of knowledge for $\mathcal{R}_{\text{KC}}$, obtained by applying the Fiat–Shamir transform to the Σ -protocol Σ_{KC} (Section A.4, Lemma 2, Figure 5); and
5. Π_{rekey} is a NIZK proof of knowledge for $\mathcal{R}_{\text{rekey}}$, a batched Chaum–Pedersen [CP93] discrete-log-equality statement obtained by applying the Fiat–Shamir transform to the Σ -protocol Σ_{rekey} (Section A.5, Lemma 3, Figure 6).

The well-formedness of an encrypted amount and of an auditor key encapsulation that the ledger checks on-chain (Section 5) are conjunctions of these base proofs over shared limb commitments; the binding argument gluing the shared openings, and the resulting discrete-log reductions, are carried out in the simulator analysis (Section 6.1). In the algorithms of Section 5, the Π_{EG} instances sharing one public key are carried as a single witness-folded proof $\Pi_{\text{EG}}^{\otimes}$ (Appendix B); in particular, the transfer’s sender-side proof γ_S folds the four new-balance limbs together with the aggregate sender-side ciphertext $\hat{A}_S$, whose commitment the ledger reconstructs from the receiver commitments. Lemma 4 shows the folded proof is a NIZK proof of knowledge for the conjunction relation, so the per-ciphertext analysis below applies unchanged.

A.1 Domain Separation

Goals. Domain separators serve two purposes. Practically, they prevent a proof produced by one user or protocol from being reused in another. In the analysis, they differentiate the random oracles of different parties, so that a simulator can program the oracle for honest parties and extract inputs from corrupted ones.

A unique domain separator should therefore be associated with the high-level protocol and the user. Subprotocol identifiers are optional, since the simulated execution may use a single ideal ZK functionality, and hence a single random oracle, per party.

For the security of the Fiat–Shamir transform, the inputs to the random oracle should include the instance, the group parameters (if not fixed), additional public information (*aad*), and the first message of the interactive ZK protocol.

Implementation. The sigma-protocol NIZKs each use a unique 21-byte session identifier *sid* as an additional input to their Fiat–Shamir hash to ensure domain separation between protocol instances and prevent cross-instance attacks. The Bulletproofs range proofs are bound to the same session identifier format. The session identifier is built in two steps. At account registration the 20-byte session identifier basis is fixed to the first 20 bytes of the user’s per-token account object address,

$$\text{base} = \text{addr}(\text{TokenAccount}\langle T \rangle)[0..20],$$

where $\text{addr}(\text{TokenAccount}\langle T \rangle)$ is deterministically derived by the ledger from the user’s account object and the token type T . The generators G and H are fixed Ristretto255 constants and are not part of the *sid*; the implementation instead prepends their encodings to every Fiat–Shamir hash as a defense-in-depth measure (item 4 below). To form the *sid*, this basis is completed by appending a single-byte protocol identifier b characterizing the subprotocol, $\text{sid} = \text{base} \parallel b$, where, for example, $b = 0x01$ for Π_{DDH} . The protocol byte is used for tightness of the reduction, not for soundness.

Necessity of each hash input. Each Fiat-Shamir challenge has the form $c = H(\text{sid}, G, H, \text{stmt}, \text{first_msg})$ with $\text{sid} = \text{addr}(\text{TokenAccount}\langle T \rangle)[0..20] \parallel b$. We classify each input by the cost of removing it.

1. *Attack without it.* The statement and first-message components in every NIZK: removing any of them breaks Fiat-Shamir directly, by letting the prover choose the omitted field after seeing c .
2. *No known attack, but needed for the proof.* The account address is prepended to every NIZK hash, binding each proof to the account that produced it and, through the derivation from the UC session identifier described in Section 6.1, to the session (Corollary 4). Some NIZKs, such as Π_{range}^{16} , are not bound to a user public key by their statement alone; without the address, a corrupted account could replay an honest account’s proof on the same statement, for which the simulator has no witness to extract, and the per-account simulation-soundness/extractability hops would fail.
3. *Only loosens the reduction.* The protocol byte b keeps the five NIZKs’ oracle slices disjoint, making the per-NIZK hops of Theorem 3 modular; dropping it costs a union bound over the statements ($Q^2/2^\lambda$, Q the total honest proofs).
4. *Redundant, but kept as defense-in-depth.* The generators G, H are fixed Ristretto255 constants whose discrete-log relationship is already pinned by the public parameters, so hashing them strengthens no reduction; the implementation nevertheless prepends their encodings to every Fiat-Shamir hash as defense-in-depth against a deployment that misconfigures the generators. The protocol figures in this paper omit these constant inputs for brevity.

No per-invocation sub-session label. The sid defined above is per-protocol-and-account, not per-invocation: a holder reuses the same sid across every **Transfer**, **Wrap**, **Unwrap**, and **Merge** they produce. This is intentional and does not need a UC-style sub-session identifier ssid on top. Every NIZK hash includes the statement and the Σ -protocol first message, both of which an honest holder freshly randomizes per proof. Two honest invocations under the same sid thus hash distinct inputs with overwhelming probability, so the random oracle is queried at distinct points and the statement plays the role of ssid . The bound is on honest holders only: corrupt provers may reuse randomness, but their proofs are extracted rather than simulated, and the account address keeps each account’s hash slice disjoint, so reuse cannot collide with a programmed point. Replay of an entire accepted invocation is blocked by the ledger functionality (Section 6.1).

A.2 DDH

The DDH proof is a zero-knowledge proof-of-knowledge for the relation $\mathcal{R}_{\text{DDH}}$ defined in Section 2: the prover demonstrates knowledge of a witness $x \in \mathbb{Z}_q$ satisfying $X = xE$ and $Y = xF$ simultaneously. The underlying interactive protocol is the classical three-move Σ -protocol of Chaum and Pedersen [CP93], which has perfect completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge. Applying the Fiat-Shamir transformation in the random oracle model yields the non-interactive variant Π_{DDH} shown in Figure 3, which we use throughout this work.

$\Pi_{\text{DDH}}.\text{Prove}(x; \text{sid}, E, F, X, Y) :$	$\Pi_{\text{DDH}}.\text{Verify}(\pi; \text{sid}, E, F, X, Y) :$
1 : $r \leftarrow \mathbb{Z}_q$	1 : $(X', Y', z) = \pi$
2 : $X' = rE, Y' = rF$	2 : $c = \text{H}(\text{sid}, E, F, X, Y, X', Y')$
3 : $c = \text{H}(\text{sid}, E, F, X, Y, X', Y')$	3 : $b_1 = (zE \stackrel{?}{=} X' + cX)$
4 : $z = r + cx$	4 : $b_2 = (zF \stackrel{?}{=} Y' + cY)$
5 : return $\pi = (X', Y', z)$	5 : return $b_1 \wedge b_2$

Figure 3: NIZK DDH Protocol Π_{DDH} ; proof size: $2\mathbb{G} + \mathbb{F}$

A.3 ElGamal PoK

We construct a NIZK PoK for the relation $\mathcal{R}_{\text{EG}}$ of a single Twisted ElGamal ciphertext (C, D) under public key pk , defined in Section 2, and prove its security. We denote its defining equations $C = rG + mH$ and $D = r\text{pk}$ with (C1) and (C2), respectively. We analyze Σ_{EG} , the public-coin three-move Σ -protocol whose Fiat–Shamir form is shown in Figure 4: the interactive protocol replaces the hash-derived challenge with a verifier-sampled $c \in \mathbb{Z}_q$. Applying the Fiat–Shamir (FS) transformation then yields $\Pi_{\text{EG}} = \text{FS}[\Sigma_{\text{EG}}]$, a NIZK proof-of-knowledge for $\mathcal{R}_{\text{EG}}$ in the random oracle model. We refer to the two verifier checks in Figure 4 as (V1) and (V2), respectively.

Lemma 1. *The protocol Σ_{EG} is a public-coin 3-move Σ -protocol for the relation $\mathcal{R}_{\text{EG}}$.*

Proof. To prove the lemma, we show completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge.

Perfect Completeness. For an honest prover, the verification equations (V1) and (V2) hold with probability 1 since:

- $z_1G + z_2H = (r_1 + cr)G + (r_2 + cm)H = r_1G + r_2H + c(rG + mH) = C' + cC$
- $z_1\text{pk} = (r_1 + cr)\text{pk} = r_1\text{pk} + cr\text{pk} = D' + cD$

Thus, completeness holds.

2-Special Soundness. Given two accepting transcripts (C', D', c, z_1, z_2) and (C', D', c', z'_1, z'_2) with $c \neq c'$, we show that there exists a PPT extractor Ext that can extract a witness w satisfying (C1) and (C2). Since both transcripts satisfy (V2), we obtain $(z_1 - z'_1)\text{pk} = (c - c')D$ by subtraction. Since $c \neq c'$, the value $c - c'$ is invertible in $\mathbb{Z}_q$ and the scalar $r = (z_1 - z'_1)/(c - c')$ is well-defined and satisfies $D = r\text{pk}$. This establishes (C2). Further, since both transcripts satisfy (V1), we obtain $(z_1 - z'_1)G + (z_2 - z'_2)H = (c - c')C$ by subtraction. Substituting $(z_1 - z'_1) = r(c - c')$ and dividing by $(c - c')$, we obtain $rG + ((z_2 - z'_2)/(c - c'))H = C$, which uniquely determines $m = (z_2 - z'_2)/(c - c')$ such that $C = rG + mH$. This establishes (C1) and overall completes witness extraction, showing that 2-special soundness holds.

Perfect Special Honest-Verifier Zero-Knowledge. Assuming the verifier is honest, we show that there exists a PPT simulator $\mathcal{S}$ that, given the statement s and the challenge c in advance, produces a transcript (C', D', c, z_1, z_2) that is identically distributed to that of an honest prover. The simulator starts by sampling values $c, z_1, z_2 \in \mathbb{Z}_q$ uniformly at random. It then sets $C' = z_1G + z_2H - cC$ and $D' = z_1\text{pk} - cD$. In an honest execution, the responses z_1 and z_2 are uniformly distributed over

$\Pi_{\text{EG}}.\text{Prove}(m, r; \text{sid}, (C, D), \text{pk}) :$	$\Pi_{\text{EG}}.\text{Verify}(\pi; \text{sid}, (C, D), \text{pk}) :$
1 : $r_1, r_2 \leftarrow \mathbb{Z}_q$	1 : $(C', D', z_1, z_2) = \pi$
2 : $C' = r_1 G + r_2 H, D' = r_1 \text{pk}$	2 : $c = H(\text{sid}, \text{pk}, C, D, C', D')$
3 : $c = H(\text{sid}, \text{pk}, C, D, C', D')$	3 : $b_1 = (z_1 G + z_2 H \stackrel{?}{=} C' + cC)$
4 : $z_1 = r_1 + cr, z_2 = r_2 + cm$	4 : $b_2 = (z_1 \text{pk} \stackrel{?}{=} D' + cD)$
5 : return $\pi = (C', D', z_1, z_2)$	5 : return $b_1 \wedge b_2$

Figure 4: NIZK PoK Protocol for Well-formedness of Twisted ElGamal Encryption

$\mathbb{Z}_q$ since r_1, r_2 are uniform. Given uniform z_1, z_2 and a fixed c , the values C' and D' are uniquely determined and their distribution in the simulation matches the honest execution. Therefore, the simulated transcript is distributed identically to the honest one, and thus, special honest-verifier zero-knowledge holds. $\square$

A.4 Key Consistency

The key-consistency proof lets a holder verifiably encrypt the eight 32-bit limbs of its secret key towards an auditor list $\mathcal{J} = (\text{pk}_{A_1}, \dots, \text{pk}_{A_n})$ using multi-recipient Twisted ElGamal encryption, while certifying that the encrypted limbs reconstruct sk_U with $\text{pk}_U = \text{sk}_U G$. We construct a NIZK PoK for the well-formedness relation for multi-recipient Twisted ElGamal encryption $\mathcal{R}_{\text{KC}}$, defined in Section 2, and prove its security. We denote its three defining conditions $D_{ij} = r_i \text{pk}_{A_j}$, $C_i = r_i G + u_i H$, and $\text{pk}_U = (\sum_{i=0}^7 2^{32i} u_i) G$ as (C1), (C2), and (C3), respectively. We analyze Σ_{KC} , the public-coin three-move Σ -protocol whose Fiat–Shamir form is shown in Figure 5: the interactive protocol replaces the hash-derived challenge with a verifier-sampled $c \in \mathbb{Z}_q$. Applying the Fiat–Shamir (FS) transformation then yields $\Pi_{\text{KC}} = \text{FS}[\Sigma_{\text{KC}}]$, a NIZK proof-of-knowledge for $\mathcal{R}_{\text{KC}}$ in the random oracle model. We refer to the three verifier checks in Figure 5 as (V1)–(V3), respectively.

The well-formedness conditions (C1) and (C2) are proved per limb, but condition (C3) is a single linear relation over all eight limbs. We therefore prove (C3) only in aggregate, committing to the single masking element $\text{pk}'_U = (\sum_{i=0}^7 b_i 2^{32i}) G$ rather than to the individual $b_i G$. This is essential for zero-knowledge: were the prover to reveal each $b_i G$ separately, a verifier could combine it with the response $z_{2i} = b_i + cu_i$ to compute $u_i G = c^{-1}(z_{2i} G - b_i G)$, and since each limb $u_i \in [0, 2^{32})$ is small, recover u_i by a discrete-logarithm search, exposing the entire witness. Committing only to the aggregate pk'_U leaves the individual $b_i G$ hidden, so no individual $u_i G$ can be isolated.

Lemma 2. *The protocol Σ_{KC} is a public-coin 3-move Σ -protocol for the relation $\mathcal{R}_{\text{KC}}$.*

Proof. To prove the lemma, we show completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge.

Perfect Completeness. For an honest prover, the verification equations (V1)–(V3) hold with probability 1 since:

- (V1) $z_{1i} \text{pk}_{A_j} = (a_i + cr_i) \text{pk}_{A_j} = a_i \text{pk}_{A_j} + cr_i \text{pk}_{A_j} = D'_{ij} + cD_{ij}$
- (V2) $z_{1i} G + z_{2i} H = (a_i + cr_i) G + (b_i + cu_i) H = a_i G + b_i H + c(r_i G + u_i H) = C'_i + cC_i$

$$\bullet \text{ (V3) } (\sum_{i=0}^7 z_{2i} 2^{32i})G = (\sum_{i=0}^7 (b_i + cu_i) 2^{32i})G = (\sum_{i=0}^7 b_i 2^{32i})G + c(\sum_{i=0}^7 u_i 2^{32i})G = \mathbf{pk}'_U + c\mathbf{pk}_U$$

Thus, completeness holds.

2-Special Soundness. Given two accepting transcripts (A, c, z) and (A, c', z') with matching commitments $A = (\{D'_{ij}, C'_i\}_{i,j}, \mathbf{pk}'_U)$, but differing challenges $c \neq c'$, and differing responses $\{z_{1i}, z_{2i}\}_i = z \neq z' = \{z'_{1i}, z'_{2i}\}_i$, we show that there exists a PPT extractor Ext that can extract a witness w satisfying (C1)–(C3).

Since both transcripts satisfy (V1), we obtain $(z_{1i} - z'_{1i})\mathbf{pk}_{A_j} = (c - c')D_{ij}$ by subtraction. Since $c \neq c'$, the value $c - c'$ is invertible in $\mathbb{Z}_q$ and the scalar $r_i = (z_{1i} - z'_{1i})/(c - c')$ is well-defined and satisfies $D_{ij} = r_i \mathbf{pk}_{A_j}$ for all i, j . This establishes (C1).

Further, since both transcripts satisfy (V2), we obtain $(z_{1i} - z'_{1i})G + (z_{2i} - z'_{2i})H = (c - c')C_i$ by subtraction. Substituting $(z_{1i} - z'_{1i}) = r_i(c - c')$ and dividing by $(c - c')$, we obtain $r_i G + ((z_{2i} - z'_{2i})/(c - c'))H = C_i$, which uniquely determines $u_i = (z_{2i} - z'_{2i})/(c - c')$ such that $C_i = r_i G + u_i H$ for all i . This establishes (C2).

Since G and H are independent generators, the representation (r_i, u_i) is unique, so each u_i is the value committed in the Pedersen commitment and used in (V3).

Finally, both transcripts satisfy (V3), which gives $(\sum_{i=0}^7 (z_{2i} - z'_{2i}) 2^{32i})G = (c - c')\mathbf{pk}_U$. Dividing by $(c - c')$ and substituting $u_i = (z_{2i} - z'_{2i})/(c - c')$ yields $(\sum_{i=0}^7 u_i 2^{32i})G = \mathbf{pk}_U$, which establishes (C3) and completes witness extraction, showing that 2-special soundness holds.

Perfect Special Honest-Verifier Zero-Knowledge. Assuming the verifier is honest, we show that there exists a PPT simulator $\mathcal{S}$ that, given the statement s and the challenge c in advance, produces a transcript (A, c, z) that is identically distributed to that of an honest prover. The simulator starts by sampling values $z_{1i}, z_{2i} \in \mathbb{Z}_q$ uniformly at random for all i . It then sets $D'_{ij} = z_{1i} \mathbf{pk}_{A_j} - cD_{ij}$ for all i, j , $C'_i = z_{1i}G + z_{2i}H - cC_i$ for all i , and the single aggregate element $\mathbf{pk}'_U = (\sum_i z_{2i} 2^{32i})G - c\mathbf{pk}_U$. By construction the simulated transcript satisfies (V1)–(V3). In an honest execution, the responses z_{1i}, z_{2i} are uniformly distributed over $\mathbb{Z}_q$ since a_i, b_i are uniform, matching the simulator's choice. Conditioned on $(z_{1i}, z_{2i})_i$ and the fixed challenge c , the commitment is fully determined and identical in both worlds: (V1) and (V2) force $D'_{ij} = z_{1i} \mathbf{pk}_{A_j} - cD_{ij}$ and $C'_i = z_{1i}G + z_{2i}H - cC_i$, while in the honest execution $\mathbf{pk}'_U = (\sum_i b_i 2^{32i})G = (\sum_i (z_{2i} - cu_i) 2^{32i})G = (\sum_i z_{2i} 2^{32i})G - c\mathbf{pk}_U$, which is exactly the value the simulator computes. Hence every commitment element is a deterministic function of the uniform responses and the statement, so the simulated transcript is distributed identically to the honest one, and special honest-verifier zero-knowledge holds. $\square$

By Lemma 2 and the Fiat–Shamir transform, $\Pi_{\text{KC}} = \text{FS}[\Sigma_{\text{KC}}]$ is a NIZK proof of knowledge for $\mathcal{R}_{\text{KC}}$ in the random oracle model. The on-chain key-encapsulation check pairs it with the 32-bit range proof Π_{range}^{32} of Section 2.

A.5 Batched Re-keying

Key rotation re-keys the four limb decryption handles of an account's active balance in place: each old handle D_i under $\mathbf{pk}_U$ is replaced by D'_i under the new key $\mathbf{pk}'_U$, while the limb commitments are reused unchanged (Algorithm 8). The holder knows the quotient $w = \mathbf{sk}'_U \mathbf{sk}_U^{-1}$ relating the two keys, and the same w transports the key and every handle: $\mathbf{pk}'_U = w \mathbf{pk}_U$ and $D'_i = w D_i$. The re-keying proof Π_{rekey} (Figure 6) is a NIZK proof of knowledge for the relation

$$\mathcal{R}_{\text{rekey}} = \left\{ ((\mathbf{pk}_U, \mathbf{pk}'_U, (D_i, D'_i)_{i=0}^3); w) : \mathbf{pk}'_U = w \mathbf{pk}_U \wedge D'_i = w D_i \text{ for } i = 0, \dots, 3 \right\},$$

a single Chaum–Pedersen [CP93] discrete-log-equality statement batched over the five base/image pairs $(\mathbf{pk}_U, \mathbf{pk}'_U)$ and $(D_i, D'_i)_{i=0}^3$ under one shared witness w . We analyze Σ_{rekey} , the public-coin

$$\Pi_{\text{KC}}.\text{Prove}((u_i, r_i)_{i=0}^7; \text{sid}, \text{pk}_U, (\text{pk}_{A_j})_{j=1}^n, (C_i, (D_{ij})_{j=1}^n)_{i=0}^7) :$$

- 1 : $a_i, b_i \leftarrow \mathbb{Z}_q$ for $i \in \{0, \dots, 7\}$
- 2 : $D'_{ij} = a_i \text{pk}_{A_j}, C'_i = a_i G + b_i H$ for $i \in \{0, \dots, 7\}, j \in \{1, \dots, n\}$
- 3 : $\text{pk}'_U = (\sum_{i=0}^7 b_i 2^{32i})G$
- 4 : $c = \text{H}(\text{sid}, \text{pk}_U, (\text{pk}_{A_j})_{j=1}^n, (C_i, (D_{ij})_{j=1}^n)_{i=0}^7, ((D'_{ij})_{j=1}^n, C'_i)_{i=0}^7, \text{pk}'_U)$
- 5 : $z_{1i} = a_i + cr_i, z_{2i} = b_i + cu_i$ for $i \in \{0, \dots, 7\}$
- 6 : **return** $\pi = ((D'_{ij})_{j=1}^n, C'_i, z_{1i}, z_{2i})_{i=0}^7, \text{pk}'_U$

$$\Pi_{\text{KC}}.\text{Verify}(\pi; \text{sid}, \text{pk}_U, (\text{pk}_{A_j})_{j=1}^n, (C_i, (D_{ij})_{j=1}^n)_{i=0}^7) :$$

- 1 : $((D'_{ij})_{j=1}^n, C'_i, z_{1i}, z_{2i})_{i=0}^7, \text{pk}'_U = \pi$
- 2 : $c = \text{H}(\text{sid}, \text{pk}_U, (\text{pk}_{A_j})_{j=1}^n, (C_i, (D_{ij})_{j=1}^n)_{i=0}^7, ((D'_{ij})_{j=1}^n, C'_i)_{i=0}^7, \text{pk}'_U)$
- 3 : $b_1 = (z_{1i} \text{pk}_{A_j} \stackrel{?}{=} D'_{ij} + c D_{ij})$
- 4 : $b_2 = (z_{1i} G + z_{2i} H \stackrel{?}{=} C'_i + c C_i)$
- 5 : $b_3 = ((\sum_{i=0}^7 z_{2i} 2^{32i})G \stackrel{?}{=} \text{pk}'_U + c \text{pk}_U)$
- 6 : **return** $b_1 \wedge b_2 \wedge b_3$

Figure 5: NIZK PoK Protocol for Well-formedness of Multi-recipient Twisted ElGamal Encryption

three-move Σ -protocol whose Fiat–Shamir form is shown in Figure 6: the interactive protocol replaces the hash-derived challenge with a verifier-sampled $c \in \mathbb{Z}_q$. We refer to the verifier check on the key pair as $(V\star)$ and to the four checks on the handle pairs as (Vi) for $i \in \{0, \dots, 3\}$.

Lemma 3. *The protocol Σ_{rekey} is a public-coin 3-move Σ -protocol for the relation $\mathcal{R}_{\text{rekey}}$.*

Proof. To prove the lemma, we show completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge.

Perfect Completeness. For an honest prover the response is $z = s + cw$, and using $\text{pk}'_U = w \text{pk}_U$ and $D'_i = w D_i$ the verification equations hold with probability 1 since:

- $(V\star) \ z \text{pk}_U = (s + cw) \text{pk}_U = s \text{pk}_U + c(w \text{pk}_U) = P' + c \text{pk}'_U$
- $(Vi) \ z D_i = (s + cw) D_i = s D_i + c(w D_i) = D'_i + c D'_i \text{ for } i \in \{0, \dots, 3\}$

Thus, completeness holds.

2-Special Soundness. Given two accepting transcripts $(P', (D'_i)_{i=0}^3, c, z)$ and $(P', (D'_i)_{i=0}^3, c', z')$ with matching first message but differing challenges $c \neq c'$, we show that there exists a PPT extractor Ext that can extract a witness w satisfying the relation. Since both transcripts satisfy $(V\star)$, subtracting $z' \text{pk}_U = P' + c' \text{pk}'_U$ from $z \text{pk}_U = P' + c \text{pk}'_U$ gives $(z - z') \text{pk}_U = (c - c') \text{pk}'_U$. Since $c \neq c'$, the value $c - c'$ is invertible in $\mathbb{Z}_q$ and the scalar $w = (z - z')/(c - c')$ is well-defined and satisfies $\text{pk}'_U = w \text{pk}_U$. Applying the same subtraction to each (Vi) yields $(z - z') D_i = (c - c') D'_i$, hence $D'_i = w D_i$ for the same scalar w , for all $i \in \{0, \dots, 3\}$. Thus Ext outputs a single w satisfying every clause of $\mathcal{R}_{\text{rekey}}$, and 2-special soundness holds.

$\Pi_{\text{rekey}}.\text{Prove}(w; \text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3) :$ 1 : $s \leftarrow \mathbb{Z}_q$ 2 : $P' = s \text{pk}_U, \quad D''_i = s D_i \text{ for } i \in \{0, \dots, 3\}$ 3 : $c = \text{H}(\text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3, P', (D''_i)_{i=0}^3)$ 4 : $z = s + c w$ 5 : return $\pi = (P', (D''_i)_{i=0}^3, z)$
$\Pi_{\text{rekey}}.\text{Verify}(\pi; \text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3) :$ 1 : $(P', (D''_i)_{i=0}^3, z) = \pi$ 2 : $c = \text{H}(\text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3, P', (D''_i)_{i=0}^3)$ 3 : $b_\star = (z \text{pk}_U \stackrel{?}{=} P' + c \text{pk}'_U)$ 4 : $b_i = (z D_i \stackrel{?}{=} D''_i + c D'_i) \text{ for } i \in \{0, \dots, 3\}$ 5 : return $b_\star \wedge b_0 \wedge b_1 \wedge b_2 \wedge b_3$

Figure 6: NIZK batched re-keying DDH protocol Π_{rekey} (shared witness over five base/image pairs); proof size: $5\mathbb{G} + \mathbb{F}$.

Perfect Special Honest-Verifier Zero-Knowledge. Assuming the verifier is honest, we show that there exists a PPT simulator $\mathcal{S}$ that, given the statement s and the challenge c in advance, produces a transcript $(P', (D''_i)_{i=0}^3, c, z)$ that is identically distributed to that of an honest prover. The simulator samples $z \in \mathbb{Z}_q$ uniformly at random and sets $P' = z \text{pk}_U - c \text{pk}'_U$ and $D''_i = z D_i - c D'_i$ for all i . By construction the simulated transcript satisfies (V $\star$) and (Vi). In an honest execution the response $z = s + c w$ is uniformly distributed over $\mathbb{Z}_q$ since s is uniform, matching the simulator's choice. Conditioned on z and the fixed challenge c , the first message $(P', (D''_i)_{i=0}^3)$ is uniquely determined by the verification equations and is identical in both worlds. Therefore, the simulated transcript is distributed identically to the honest one, and special honest-verifier zero-knowledge holds. $\square$

By Lemma 3 and the Fiat-Shamir transform, $\Pi_{\text{rekey}} = \text{FS}[\Sigma_{\text{rekey}}]$ is a NIZK proof of knowledge for $\mathcal{R}_{\text{rekey}}$ in the random oracle model. The on-chain key-rotation check (Algorithm 8) pairs it with the assertion $\text{pk}'_U \neq \mathcal{O}$ and a fresh auditor encapsulation under the new key.

$\Pi_{\mathbf{B}}^{\otimes}.\text{Prove}((\mathbf{x}_j)_j; \text{sid}, \mathbf{B}, (\mathbf{y}_j)_j) :$	$\Pi_{\mathbf{B}}^{\otimes}.\text{Verify}(\pi; \text{sid}, \mathbf{B}, (\mathbf{y}_j)_j) :$
1 : $\mathbf{s} \leftarrow \mathbb{Z}_q^m; \mathbf{T} = \mathbf{B} \mathbf{s} \in \mathbb{G}^p$	1 : $(\mathbf{T}, \mathbf{z}) = \pi$
2 : $c = \text{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T})$	2 : $c = \text{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T})$
3 : $\mathbf{z} = \mathbf{s} + \sum_{j=1}^n c^j \mathbf{x}_j \in \mathbb{Z}_q^m$	3 : return $\mathbf{B} \mathbf{z} \stackrel{?}{=} \mathbf{T} + \sum_{j=1}^n c^j \mathbf{y}_j$
4 : return $\pi = (\mathbf{T}, \mathbf{z})$	

Figure 7: Witness-fold NIZKPoK $\Pi_{\mathbf{B}}^{\otimes}$ for the tensor language $\mathbf{I}_n \otimes \mathbf{B}$ with block $\mathbf{B} \in \mathbb{G}^{p \times m}$; proof size $p\mathbb{G} + m\mathbb{F}$, independent of the number of instances n .

B Witness-Folding: Batching Schnorr Proofs over a Shared Base Block

Many protocol operations emit several Σ -proofs over the same public bases: a transfer proves one ElGamal consistency per receiver under a common ciphertext layout, unwrap and balance updates prove a fixed set of same-key consistencies, and so on. Written out, n such proofs over one base block cost n times a single proof. This appendix shows that instances sharing a base block collapse to one proof whose size does not grow with n .

Setup. View $\mathbb{G}$ (prime order q , written additively) as a $\mathbb{Z}_q$ -module and extend the scalar action entrywise to matrices, so a base block $\mathbf{B} \in \mathbb{G}^{p \times m}$ acts as a $\mathbb{Z}_q$ -linear map $\mathbf{B} : \mathbb{Z}_q^m \rightarrow \mathbb{G}^p, \mathbf{x} \mapsto \mathbf{B}\mathbf{x}$. A single Schnorr instance is a witness $\mathbf{x} \in \mathbb{Z}_q^m$ with $\mathbf{B}\mathbf{x} = \mathbf{y}$ for a public target $\mathbf{y} \in \mathbb{G}^p$; its generalized-Schnorr proof has size $p\mathbb{G} + m\mathbb{F}$.

The tensor language. Fix $\mathbf{B} \in \mathbb{G}^{p \times m}$ and take n instances sharing it: witnesses $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{Z}_q^m$ and targets $\mathbf{y}_j = \mathbf{B}\mathbf{x}_j$. Stacking them gives the block-diagonal Kronecker system

$$(\mathbf{I}_n \otimes \mathbf{B}) \mathbf{x} = \mathbf{y}, \quad \mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n) \in \mathbb{Z}_q^{nm}, \quad \mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_n) \in \mathbb{G}^{np},$$

with relation

$$\mathcal{R}_{\mathbf{B}}^{\otimes} = \{(\mathbf{B}, (\mathbf{y}_j)_{j=1}^n; (\mathbf{x}_j)_{j=1}^n) : \mathbf{y}_j = \mathbf{B}\mathbf{x}_j \text{ for all } j\}.$$

Proving the instances one by one costs $n(p\mathbb{G} + m\mathbb{F})$. The witness-fold instead combines them along their witnesses (the columns of the Kronecker system): for a challenge c , form the aggregate witness and target

$$\mathbf{x}^* = \sum_{j=1}^n c^j \mathbf{x}_j \in \mathbb{Z}_q^m, \quad \mathbf{y}^* = \sum_{j=1}^n c^j \mathbf{y}_j \in \mathbb{G}^p,$$

which satisfy $\mathbf{B}\mathbf{x}^* = \mathbf{y}^*$ by linearity. A single generalized-Schnorr proof for the aggregate instance $(\mathbf{B}, \mathbf{y}^*)$ then certifies the whole batch. Drawing c from the random oracle on the statement prevents the prover from choosing a batch the aggregate would mask. The Fiat–Shamir proof $\Pi_{\mathbf{B}}^{\otimes}$ is given in Figure 7; its size is $p\mathbb{G} + m\mathbb{F}$, independent of n .

Lemma 4 (Witness-fold is an unconditional NIZKPoK). *Let $\mathbf{B} \in \mathbb{G}^{p \times m}$ be any base block over $\mathbb{G}$ (prime order q) and H a random oracle. Then $\Pi_{\mathbf{B}}^{\otimes}$ (Figure 7) is a complete, special honest-verifier zero-knowledge, $(n+1)$ -special-sound Fiat–Shamir NIZK proof of knowledge for $\mathcal{R}_{\mathbf{B}}^{\otimes}$.*

Proof. Throughout, $\mathbf{B}$ is a fixed $\mathbb{Z}_q$ -linear map $\mathbb{Z}_q^m \rightarrow \mathbb{G}^p$.

Completeness. For the honest response $\mathbf{z} = \mathbf{s} + \sum_j c^j \mathbf{x}_j$, linearity gives $\mathbf{Bz} = \mathbf{Bs} + \sum_j c^j \mathbf{Bx}_j = \mathbf{T} + \sum_j c^j \mathbf{y}_j$, using $\mathbf{y}_j = \mathbf{Bx}_j$.

SHVZK. On challenge c , the simulator draws $\mathbf{z} \leftarrow \mathbb{Z}_q^m$, sets $\mathbf{T} := \mathbf{Bz} - \sum_j c^j \mathbf{y}_j$, and programs $\mathbf{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T}) = c$. The transcript verifies by construction. In a real run $\mathbf{z} = \mathbf{s} + \sum_j c^j \mathbf{x}_j$ is uniform on $\mathbb{Z}_q^m$ (since $\mathbf{s}$ is), and the honest $\mathbf{T} = \mathbf{Bs} = \mathbf{Bz} - \sum_j c^j \mathbf{y}_j$ is the same function of $(\mathbf{z}, \text{statement})$ as in the simulation, so the two distributions are identical.

(n+1)-special soundness. Fix a first message $\mathbf{T}$ and take $n+1$ accepting transcripts with pairwise-distinct challenges $c_0, \dots, c_n$ and responses $\mathbf{z}_0, \dots, \mathbf{z}_n \in \mathbb{Z}_q^m$. The accepting equations

$$\mathbf{Bz}_i = \mathbf{T} + \sum_{j=1}^n c_i^j \mathbf{y}_j \in \mathbb{G}^p \quad (i = 0, \dots, n)$$

state that the group elements $\mathbf{Bz}_i$ are the values at c_i of the degree- $\leq n$ polynomial whose coefficients are $\mathbf{T}, \mathbf{y}_1, \dots, \mathbf{y}_n$ ($\mathbf{T}$ is the degree-0 term). The nodes c_i are distinct, so the Vandermonde matrix $V = (c_i^j)_{0 \leq i, j \leq n}$ is invertible over $\mathbb{Z}_q$; reading off the degree- j coefficient and using linearity of $\mathbf{B}$ yields, for $1 \leq j \leq n$,

$$\mathbf{y}_j = \sum_i (V^{-1})_{ji} \mathbf{Bz}_i = \mathbf{Bx}_j, \quad \mathbf{x}_j := \sum_i (V^{-1})_{ji} \mathbf{z}_i \in \mathbb{Z}_q^m.$$

Thus $(\mathbf{x}_j)_{j=1}^n$ is a witness for $\mathcal{R}_{\mathbf{B}}^\otimes$. The extractor computes it by solving the $\mathbb{Z}_q$ -Vandermonde system on the responses, and its correctness uses only that $\mathbf{B}$ is $\mathbb{Z}_q$ -linear (evaluated forward, as in $\sum_i (V^{-1})_{ji} \mathbf{Bz}_i = \mathbf{B} \sum_i (V^{-1})_{ji} \mathbf{z}_i$). No discrete log is ever taken, so the argument is unaffected by any discrete-log relation among the entries of $\mathbf{B}$. $\square$

Application: DDH tuples ($2\mathbb{G} + \mathbb{F}$). Take $\mathbf{B} = \begin{pmatrix} E \\ F \end{pmatrix}$ ($p = 2, m = 1$) and targets $\mathbf{y}_j = (X_j, Y_j)^\top$ with $X_j = x_j E, Y_j = x_j F$. Writing $\mathbf{T} = (X', Y')^\top = (sE, sF)^\top$ and $\mathbf{z} = z$, the proof $\pi = (X', Y', z) \in 2\mathbb{G} + \mathbb{F}$ has the two checks

$$zE \stackrel{?}{=} X' + \sum_j c^j X_j \quad (\text{V1}), \quad zF \stackrel{?}{=} Y' + \sum_j c^j Y_j \quad (\text{V2}).$$

Lemma 4 extracts a single scalar per tuple, $x_j = \sum_i (V^{-1})_{ji} z_i$, which satisfies the whole relation $\mathbf{Bx}_j = \mathbf{y}_j$, that is $X_j = x_j E$ (from V1) and $Y_j = x_j F$ (from V2) with the one common x_j . Each (X_j, Y_j) is therefore a genuine DDH tuple, and this holds even when the prover knows $\log_E F$.

Application: Same-key ElGamal ($2\mathbb{G} + 2\mathbb{F}$). Take $\mathbf{B} = \text{eg}(\text{pk}) = \begin{pmatrix} G & H \\ \text{pk} & 0 \end{pmatrix}$ ($p = 2, m = 2$) and n ciphertexts (C_j, D_j) under one key pk , with witnesses $\mathbf{x}_j = (r_j, v_j)$. Writing $\mathbf{T} = (C', D')^\top = (aG + bH, a\text{pk})^\top$ and $\mathbf{z} = (z_1, z_2)$,

$$a, b \leftarrow \mathbb{Z}_q; \quad z_1 = a + \sum_j c^j r_j, \quad z_2 = b + \sum_j c^j v_j; \quad c = \mathbf{H}(\text{sid}, \text{pk}, (C_j, D_j)_j, C', D'); \quad \pi = (C', D', z_1, z_2),$$

$$z_1 G + z_2 H \stackrel{?}{=} C' + \sum_j c^j C_j \quad (\text{V1}), \quad z_1 \text{pk} \stackrel{?}{=} D' + \sum_j c^j D_j \quad (\text{V2}),$$

of size $2\mathbb{G} + 2\mathbb{F}$. The extractor returns $r_j = \sum_i (V^{-1})_{ji} z_1^{(i)}$ and $v_j = \sum_i (V^{-1})_{ji} z_2^{(i)}$ from (V1), and $D_j = r_j \text{pk}$ from (V2) via the shared $z_1^{(i)}$. Because both checks are kept, the proof stays sound even though a holder knows its own $\text{sk} = \log_G \text{pk}$; this is the base relation covered by Lemma 4.

Algorithm	Witness-fold batches	Standalone	Unoptimized size	Witness-fold optimized
1. Register (Alg. 1)	—	Π_{KC}	$(8n_A+9)\mathbb{G} + 16\mathbb{F}$	$(8n_A+9)\mathbb{G} + 16\mathbb{F}$
2. Auditor update (Alg. 2)	—	—	—	—
3. Wrap	—	—	—	—
4–5. Transfer (Alg. 4, 5)	$\mathbf{eg}_{\text{pk}_S}^{(5)}, \mathbf{eg}_{\text{pk}_{R_k}}^{(4)} \times K$	π_{bal}	$(8K+12)\mathbb{G} + (8K+11)\mathbb{F}$	$(2K+4)\mathbb{G} + (2K+3)\mathbb{F}$
6. Unwrap (Alg. 6)	$\mathbf{eg}_{\text{pk}_U}^{(4)}$	π_{bal}	$10\mathbb{G} + 9\mathbb{F}$	$4\mathbb{G} + 3\mathbb{F}$
7. Merge / update bal. (Alg. 7)	$\mathbf{eg}_{\text{pk}_U}^{(4)}$	π_{eq}	$10\mathbb{G} + 9\mathbb{F}$	$4\mathbb{G} + 3\mathbb{F}$
8. Key rotation (Alg. 8)	—	$\Pi_{\text{rekey}} + \Pi_{\text{KC}}$	$(8n_A+14)\mathbb{G} + 17\mathbb{F}$	$(8n_A+14)\mathbb{G} + 17\mathbb{F}$

Table 1: Witness-fold optimization per operation. Consistency proofs sharing one key form a tensor $\mathbf{I}_t \otimes \mathbf{eg}(\text{pk})$ and batch to $2\mathbb{G} + 2\mathbb{F}$ independent of their count (Lemma 4); this holds even though holders know their own sk , since both ElGamal checks are retained. Each balance/equality DDH is a lone tuple and stays at $2\mathbb{G} + \mathbb{F}$. Π_{KC} and Π_{rekey} are not same-key $\mathbf{eg}$ tensors and are unchanged. K is the number of transfer receivers and n_A the number of auditors; the unoptimized size sums the constituent proofs (Appendix A). Range proofs are unaffected by the fold and excluded from both size columns.

One batched proof per operation. Table 1 applies the witness-fold to each operation’s proof bundle. Consistency proofs sharing one key form a tensor $\mathbf{I}_t \otimes \mathbf{eg}(\text{pk})$ and batch to $2\mathbb{G} + 2\mathbb{F}$ regardless of their count t (Lemma 4); we write $\mathbf{eg}_{\text{pk}}^{(t)}$ for such a batch. The balance and equality proofs are lone DDH tuples (one per operation), so there is nothing to batch and each stays at its standalone $2\mathbb{G} + \mathbb{F}$. Π_{KC} and Π_{rekey} do not admit witness-folding and are left unchanged.